LinkedIn Learning: Data Quality Upstream Project Walkthrough

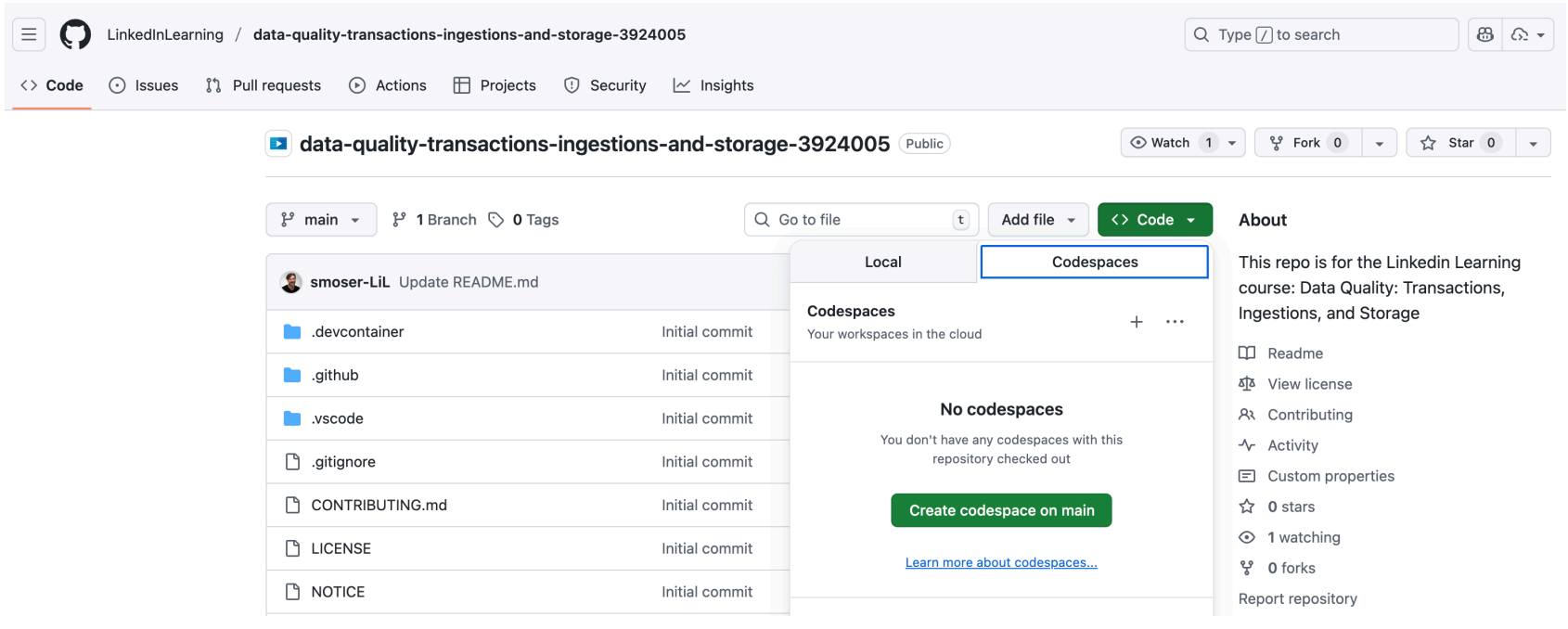
Setup and Background

Github Codespaces

> [!NOTE]
> I highly suggest starting GitHub Codespaces right away and let it build in the background while discuss the project and mock scenario. Note that the sandbox environment will take a few minutes as it's building an entire data platform and running it end-to-end to ensure it works properly.

Starting up codespaces is simple:

1. Click the green "< > Code" button.
2. Select "Codespaces."
3. Press the "+" button.
4. Wait for the environment to build for a couple minutes.
5. Start coding with a real-world environment in the browser!



Scenario:

In this project you are a data engineer that's been tasked with exploring data quality enhancements to the company's new data platform. Given that the company is relatively early in their data maturity, you want to first understand what could potentially break the platform and provide a product requirements document (PRD) for your tech lead's review.

To support your work, an isolated sandbox environment has been created with the data platform and a sample of the data. By the end of the project, you should have a PRD with the following:

- An overview of the data platform architecture.
- Potential gaps in the data platform for the following areas:
    1. Ingestion into the transactional database.
    2. Transactions on the transactional database.
    3. Replication from the transactional database to the data lakehouse.
- Suggestions on improvements, their respective tradeoffs, and results of your implementation tests.
- Remaining questions that should be addressed but are outside the scope of this project.

Data:

This project utilizes two public government datasets sourced from NYC Open Data:

1. NYC Parking Violations Issued - Fiscal Year 2023 (https://data.cityofnewyork.us/City-Government/Parking-Violations-Issued-Fiscal-Year-2023/pvqr-7yc4)

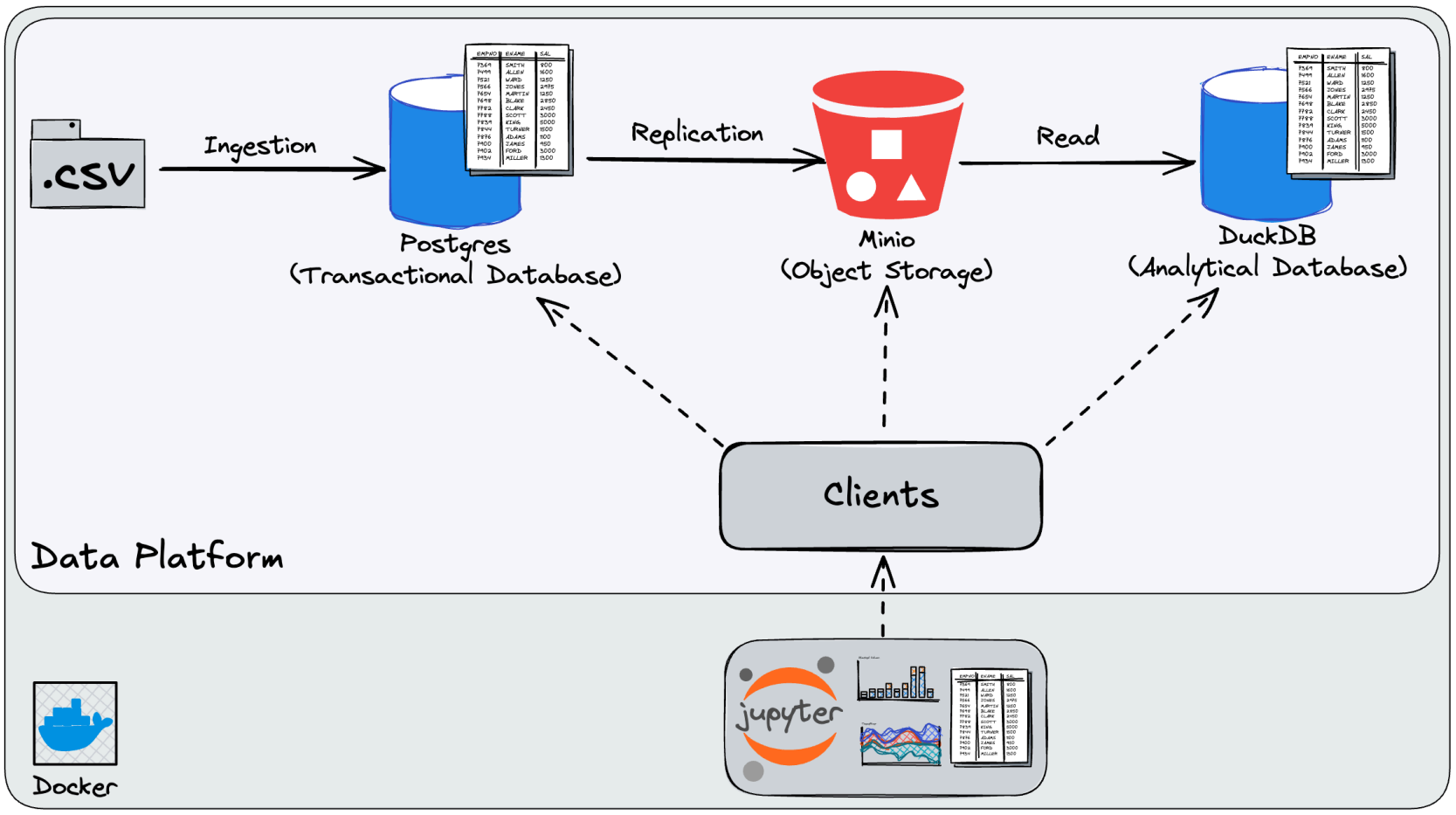
> Parking Violations Issuance datasets contain violations issued during the respective fiscal year. The Issuance datasets are not updated to reflect violation status, the information only represents the violation(s) at the time they are issued. Since appearing on an issuance dataset, a violation may have been paid, dismissed via a hearing, statutorily expired, or had other changes to its status. To see the current status of outstanding parking violations, please look at the Open Parking & Camera Violations dataset.

Note that this dataset is rather large for this project, so we will use a sample of 100K records.

2. NYC Department of Finance Parking Violation Codes (https://data.cityofnewyork.us/Transportation/DOF-Parking-Violation-Codes/ncbg-6agr)

> This dataset defines the parking violation codes in New York City and lists the fines.

Data Platform Architecture



Project Structure:

```
linkedin-learning-dq-upstream/
├── course_assets/                                  # Course materials and documentation
│   ├── product_requirements_document_template.md   # PRD template for course projects
│   ├── product_requirements_document_complete.md   # PRD completed for course projects
│   ├── project_walkthrough_complete.ipynb          # Course walkthrough
│   └── project_walkthrough_images/                 # Course walkthrough images
├── data_platform/                                  # Data platform code
│   ├── clients/                                    # Database and service clients
│   │   ├── postgres_client.py                      # PostgreSQL connection and operations
│   │   ├── minio_client.py                         # MinIO client for object storage
│   │   └── duckdb_client.py                        # DuckDB connection and operations
│   ├── config/                                     # Configuration management
│   │   ├── postgres_config.py                      # PostgreSQL connection configuration
│   │   ├── minio_config.py                         # MinIO connection configuration
│   │   └── duckdb_config.py                        # DuckDB connection configuration
│   ├── data/                                       # Data files and schemas
│   │   ├── clean_data/                             # Clean CSV data files
│   │   │   ├── dof_parking_violation_codes.csv     # Clean parking violation codes CSV file
│   │   │   └── parking_violations_issued...sample.csv # Clean parking violations issued CSV file
│   │   ├── dirty_data/                             # Dirty CSV data files
│   │   │   ├── dirty_dof_parking_violation_codes.csv  # Dirty parking violation codes CSV file
│   │   │   └── dirty_parking_violations_issued.csv    # Dirty parking violations issued CSV file
│   │   └── schemas/                                # Data schema definitions JSON files
│   │       ├── parking_violation_codes.json        # Parking violation codes schema JSON file
│   │       └── parking_violations_issued.json      # Parking violations issued schema JSON file
│   ├── etl/                                        # ETL/ELT pipeline implementations
│   │   ├── batch_postgres_minio_etl.py             # ETL: PostgreSQL → MinIO
│   │   ├── batch_postgres_minio_wap_etl.py         # ETL: PostgreSQL → MinIO with WAP pattern
│   │   └── batch_minio_duckdb_elt.py               # ELT: MinIO → DuckDB
│   ├── ingestion/                                  # Data ingestion modules
│   │   └── postgres_ingestion_csv.py               # CSV file ingestion to PostgreSQL
│   ├── lakehouse/                                  # DuckDB data lakehouse
│   │   └── lakehouse.db                            # DuckDB persistent database
│   ├── scripts/                                    # Executable scripts
│   │   ├── run_ingestion.py                        # Data ingestion runner
│   │   ├── run_wap_etl.py                          # Combined ETL+ELT runner with WAP pattern
│   │   └── run_etl.py                              # Combined ETL+ELT runner
│   ├── transactions/                               # Transaction and concurrency modules
│   │   └── concurrency_simulator.py                # Transaction concurrency simulation
│   └── setup_data_platform.sh                      # Data platform setup script
└── project_walkthrough.ipynb                       # Main project walkthrough
```

Imports and Setup

In [1…

```
import logging
from data_platform.clients.postgres_client import PostgresClient
from data_platform.clients.minio_client import MinioClient
from data_platform.clients.duckdb_client import DuckdbClient

from data_platform.ingestion.postgres_ingestion_csv import PostgresIngestionCSV
from data_platform.transactions.concurrency_simulator import TransactionSimulator
from data_platform.etl.batch_postgres_minio_wap_etl import BatchPostgresMinioWriteAuditPublishETL

logging.basicConfig(level=logging.DEBUG, format='%(levelname)s - %(name)s - %(message)s')

postgres = PostgresClient()
minio = MinioClient()

simulator = TransactionSimulator()
```

Chapter 1 - Data Ingestion

Our data is already ingested from the setup of the sandbox environment. While we could look at the setup logs, it's easier to just run the data ingestion pipeline again and see the logs output.

Please note the following line that prints out logs at the debugging level within the notebook cells:

`logging.basicConfig(level=logging.DEBUG, format='%(levelname)s - %(name)s - %(message)s')`
In the below cell, let's run the following bash code and review the logs:

1.1 - Initial Ingestion Into PostgreSQL

In [2…

```
!python3 data_platform/scripts/run_ingestion.py
```

```
INFO - __main__ - Starting ingestion for parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/clean_data/dof
_parking_violation_codes.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violation_codes.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['CODE', 'DEFINITION', 'Manhat
tan\xa0 96th St. & below', 'All Other Areas'] -> ['code', 'definition', 'manhattan_96th_st_below', 'all_other
_areas']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 97 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violation_codes with 4 column
s
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
on_codes:

            CREATE TABLE IF NOT EXISTS parking_violation_codes (
                code INTEGER PRIMARY KEY, definition TEXT, manhattan_96th_st_below INTEGER, all_other_areas I
NTEGER, pg_created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
on_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 97 rows into parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['code', 'definitio
n', 'manhattan_96th_st_below', 'all_other_areas']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['code']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 97 rows in parking_viol
ation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 97 rows
INFO - __main__ - Completed ingestion for parking_violation_codes
INFO - __main__ - Starting ingestion for parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/clean_data/par
king_violations_issued_fiscal_year_2023_sample.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violations_issued.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['Summons Number', 'Registrati
on State', 'Plate Type', 'Issue Date', 'Violation Code', 'Vehicle Body Type', 'Vehicle Make', 'Issuing Agenc
y', 'Vehicle Expiration Date', 'Violation Location', 'Violation Precinct', 'Issuer Precinct', 'Issuer Code',
'Issuer Command', 'Issuer Squad', 'Violation Time', 'Time First Observed', 'Violation County', 'Violation In
Front Of Or Opposite', 'Date First Observed', 'Law Section', 'Sub Division', 'Violation Legal Code', 'Days Pa
rking In Effect    ', 'From Hours In Effect', 'To Hours In Effect', 'Vehicle Color', 'Unregistered Vehicle?',
'Vehicle Year', 'Meter Number', 'Feet From Curb', 'No Standing or Stopping Violation', 'Hydrant Violation',
'Double Parking Violation'] -> ['summons_number', 'registration_state', 'plate_type', 'issue_date', 'violatio
n_code', 'vehicle_body_type', 'vehicle_make', 'issuing_agency', 'vehicle_expiration_date', 'violation_locatio
n', 'violation_precinct', 'issuer_precinct', 'issuer_code', 'issuer_command', 'issuer_squad', 'violation_tim
e', 'time_first_observed', 'violation_county', 'violation_in_front_of_or_opposite', 'date_first_observed', 'l
aw_section', 'sub_division', 'violation_legal_code', 'days_parking_in_effect', 'from_hours_in_effect', 'to_ho
urs_in_effect', 'vehicle_color', 'unregistered_vehicle', 'vehicle_year', 'meter_number', 'feet_from_curb', 'n
o_standing_or_stopping_violation', 'hydrant_violation', 'double_parking_violation']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 100000 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violations_issued with 34 col
umns
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
ons_issued:

            CREATE TABLE IF NOT EXISTS parking_violations_issued (
                summons_number BIGINT PRIMARY KEY, registration_state TEXT, plate_type TEXT, issue_date DATE,
violation_code INTEGER, vehicle_body_type TEXT, vehicle_make TEXT, issuing_agency TEXT, vehicle_expiration_da
te TEXT, violation_location TEXT, violation_precinct INTEGER, issuer_precinct INTEGER, issuer_code INTEGER, i
ssuer_command TEXT, issuer_squad TEXT, violation_time TEXT, time_first_observed TEXT, violation_county TEXT,
violation_in_front_of_or_opposite TEXT, date_first_observed TEXT, law_section INTEGER, sub_division TEXT, vio
lation_legal_code TEXT, days_parking_in_effect TEXT, from_hours_in_effect TEXT, to_hours_in_effect TEXT, vehi
cle_color TEXT, unregistered_vehicle TEXT, vehicle_year INTEGER, meter_number TEXT, feet_from_curb INTEGER, n
o_standing_or_stopping_violation TEXT, hydrant_violation TEXT, double_parking_violation TEXT, pg_created_at T
IMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
ons_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
s_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 100000 rows into parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['summons_number', 'r
egistration_state', 'plate_type', 'issue_date', 'violation_code', 'vehicle_body_type', 'vehicle_make', 'issui
ng_agency', 'vehicle_expiration_date', 'violation_location', 'violation_precinct', 'issuer_precinct', 'issuer
_code', 'issuer_command', 'issuer_squad', 'violation_time', 'time_first_observed', 'violation_county', 'viola
tion_in_front_of_or_opposite', 'date_first_observed', 'law_section', 'sub_division', 'violation_legal_code',
'days_parking_in_effect', 'from_hours_in_effect', 'to_hours_in_effect', 'vehicle_color', 'unregistered_vehicl
e', 'vehicle_year', 'meter_number', 'feet_from_curb', 'no_standing_or_stopping_violation', 'hydrant_violatio
n', 'double_parking_violation']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['summons_number']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 100000 rows in parking_
violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 100000 rows
```

```
INFO - __main__ - Completed ingestion for parking_violations_issued
INFO - __main__ - Found 2 public tables
INFO - __main__ - Public tables dataframe:
               table_name
  parking_violation_codes
parking_violations_issued
```

Now let's create a simple report with both ingested data assets to serve as a baseline for our data quality checks with the following query. Remember, we are less worried about the data itself and instead of the robustness of the data platform and aligning with our expectations of the data.

In [3…

```
sql_query = """
SELECT
    parking_violation_codes.code,
    parking_violation_codes.definition,
    COUNT(parking_violations_issued.violation_code) AS ticket_count
FROM
    parking_violation_codes
JOIN
    parking_violations_issued ON
    parking_violation_codes.code = parking_violations_issued.violation_code
GROUP BY
    parking_violation_codes.code,
    parking_violation_codes.definition
ORDER BY
    ticket_count DESC
LIMIT 10;
"""

postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Out[3…

	code	definition	ticket_count
0	36	PHTO SCHOOL ZN SPEED VIOLATION	37064
1	21	NO PARKING-STREET CLEANING	12149
2	38	FAIL TO DSPLY MUNI METER RECPT	6482
3	14	NO STANDING-DAY/TIME LIMITS	5047
4	7	FAILURE TO STOP AT RED LIGHT	4055
5	20	NO PARKING-DAY/TIME LIMITS	3949
6	40	FIRE HYDRANT	3902
7	71	INSP. STICKER-EXPIRED/MISSING	3655
8	5	BUS LANE VIOLATION	3477
9	70	REG. STICKER-EXPIRED/MISSING	2595

1.2 - Ingesting Dirty Data

Now let's intentionally ingest bad data to see how the report changes and identify what improvements we can make to the data platform.

Note that this time we are using `PostgresIngestionCSV().ingest()` directly to ingest the dirty data as the ingestion scripts only ingest the clean data.

In [4…

```
PostgresIngestionCSV().ingest(
    csv_path='data_platform/data/dirty_data/dirty_parking_violations_issued.csv',
    schema_path='data_platform/data/schemas/parking_violations_issued.json'
)

PostgresIngestionCSV().ingest(
    csv_path='data_platform/data/dirty_data/dirty_dof_parking_violation_codes.csv',
    schema_path='data_platform/data/schemas/parking_violation_codes.json'
)
```

```
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/dirty_data/dir
ty_parking_violations_issued.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violations_issued.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['Summons Number', 'Violation
Code'] -> ['summons_number', 'violation_code']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 10000 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violations_issued with 34 col
umns
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
ons_issued:

            CREATE TABLE IF NOT EXISTS parking_violations_issued (
                summons_number BIGINT PRIMARY KEY, registration_state TEXT, plate_type TEXT, issue_date DATE,
violation_code INTEGER, vehicle_body_type TEXT, vehicle_make TEXT, issuing_agency TEXT, vehicle_expiration_da
te TEXT, violation_location TEXT, violation_precinct INTEGER, issuer_precinct INTEGER, issuer_code INTEGER, i
ssuer_command TEXT, issuer_squad TEXT, violation_time TEXT, time_first_observed TEXT, violation_county TEXT,
violation_in_front_of_or_opposite TEXT, date_first_observed TEXT, law_section INTEGER, sub_division TEXT, vio
lation_legal_code TEXT, days_parking_in_effect TEXT, from_hours_in_effect TEXT, to_hours_in_effect TEXT, vehi
cle_color TEXT, unregistered_vehicle TEXT, vehicle_year INTEGER, meter_number TEXT, feet_from_curb INTEGER, n
o_standing_or_stopping_violation TEXT, hydrant_violation TEXT, double_parking_violation TEXT, pg_created_at T
IMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
ons_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
s_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 10000 rows into parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['summons_number', 'v
iolation_code']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['summons_number']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 10000 rows in parking_v
iolations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 10000 rows
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/dirty_data/dir
ty_dof_parking_violation_codes.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violation_codes.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['CODE', 'DEFINITION'] -> ['co
de', 'definition']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 1 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violation_codes with 4 column
s
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
on_codes:

            CREATE TABLE IF NOT EXISTS parking_violation_codes (
                code INTEGER PRIMARY KEY, definition TEXT, manhattan_96th_st_below INTEGER, all_other_areas I
NTEGER, pg_created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
on_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 1 rows into parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['code', 'definitio
n']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['code']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 1 rows in parking_viola
tion_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 1 rows
```

Then let's run the same report again and see the results.

In [5…
```
sql_query = """
SELECT
    parking_violation_codes.code,
    parking_violation_codes.definition,
    COUNT(parking_violations_issued.violation_code) AS ticket_count
FROM
    parking_violation_codes
JOIN
    parking_violations_issued ON
    parking_violation_codes.code = parking_violations_issued.violation_code
GROUP BY
    parking_violation_codes.code,
    parking_violation_codes.definition
ORDER BY
    ticket_count DESC
LIMIT 10;
"""

postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Out[5…

	code	definition	ticket_count
0	36	PHTO SCHOOL ZN SPEED VIOLATION	37064
1	21	NO PARKING-STREET CLEANING	12149
2	-99	TEST	10000
3	38	FAIL TO DSPLY MUNI METER RECPT	6482
4	14	NO STANDING-DAY/TIME LIMITS	5047
5	7	FAILURE TO STOP AT RED LIGHT	4055
6	20	NO PARKING-DAY/TIME LIMITS	3949
7	40	FIRE HYDRANT	3902
8	71	INSP. STICKER-EXPIRED/MISSING	3655
9	5	BUS LANE VIOLATION	3477

You should now see a row with a code of `-99` and a definition of `TEST` for 10,000 records. So despite having schema validation, we are still able to ingest bad data. In the next video, we will implement CHECK constraints to prevent this from happening.

1.3 - Implementing CHECK Constraints

Lets drop the tables and re-ingest the data to start fresh to see how we can place some data quality checks in place.

In [6…
```
sql_query = """
DROP TABLE IF EXISTS parking_violations_issued CASCADE;
DROP TABLE IF EXISTS parking_violation_codes CASCADE;
"""

postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Let's run the ingestion again.

In [7…
```
!python3 data_platform/scripts/run_ingestion.py
```

```
INFO - __main__ - Starting ingestion for parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/clean_data/dof
_parking_violation_codes.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violation_codes.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['CODE', 'DEFINITION', 'Manhat
tan\xa0 96th St. & below', 'All Other Areas'] -> ['code', 'definition', 'manhattan_96th_st_below', 'all_other
_areas']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 97 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violation_codes with 4 column
s
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
on_codes:

            CREATE TABLE IF NOT EXISTS parking_violation_codes (
                code INTEGER PRIMARY KEY, definition TEXT, manhattan_96th_st_below INTEGER, all_other_areas I
NTEGER, pg_created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
on_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 97 rows into parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['code', 'definitio
n', 'manhattan_96th_st_below', 'all_other_areas']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['code']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 97 rows in parking_viol
ation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 97 rows
INFO - __main__ - Completed ingestion for parking_violation_codes
INFO - __main__ - Starting ingestion for parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/clean_data/par
king_violations_issued_fiscal_year_2023_sample.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violations_issued.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['Summons Number', 'Registrati
on State', 'Plate Type', 'Issue Date', 'Violation Code', 'Vehicle Body Type', 'Vehicle Make', 'Issuing Agenc
y', 'Vehicle Expiration Date', 'Violation Location', 'Violation Precinct', 'Issuer Precinct', 'Issuer Code',
'Issuer Command', 'Issuer Squad', 'Violation Time', 'Time First Observed', 'Violation County', 'Violation In
Front Of Or Opposite', 'Date First Observed', 'Law Section', 'Sub Division', 'Violation Legal Code', 'Days Pa
rking In Effect    ', 'From Hours In Effect', 'To Hours In Effect', 'Vehicle Color', 'Unregistered Vehicle?',
'Vehicle Year', 'Meter Number', 'Feet From Curb', 'No Standing or Stopping Violation', 'Hydrant Violation',
'Double Parking Violation'] -> ['summons_number', 'registration_state', 'plate_type', 'issue_date', 'violatio
n_code', 'vehicle_body_type', 'vehicle_make', 'issuing_agency', 'vehicle_expiration_date', 'violation_locatio
n', 'violation_precinct', 'issuer_precinct', 'issuer_code', 'issuer_command', 'issuer_squad', 'violation_tim
e', 'time_first_observed', 'violation_county', 'violation_in_front_of_or_opposite', 'date_first_observed', 'l
aw_section', 'sub_division', 'violation_legal_code', 'days_parking_in_effect', 'from_hours_in_effect', 'to_ho
urs_in_effect', 'vehicle_color', 'unregistered_vehicle', 'vehicle_year', 'meter_number', 'feet_from_curb', 'n
o_standing_or_stopping_violation', 'hydrant_violation', 'double_parking_violation']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 100000 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violations_issued with 34 col
umns
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
ons_issued:

            CREATE TABLE IF NOT EXISTS parking_violations_issued (
                summons_number BIGINT PRIMARY KEY, registration_state TEXT, plate_type TEXT, issue_date DATE,
violation_code INTEGER, vehicle_body_type TEXT, vehicle_make TEXT, issuing_agency TEXT, vehicle_expiration_da
te TEXT, violation_location TEXT, violation_precinct INTEGER, issuer_precinct INTEGER, issuer_code INTEGER, i
ssuer_command TEXT, issuer_squad TEXT, violation_time TEXT, time_first_observed TEXT, violation_county TEXT,
violation_in_front_of_or_opposite TEXT, date_first_observed TEXT, law_section INTEGER, sub_division TEXT, vio
lation_legal_code TEXT, days_parking_in_effect TEXT, from_hours_in_effect TEXT, to_hours_in_effect TEXT, vehi
cle_color TEXT, unregistered_vehicle TEXT, vehicle_year INTEGER, meter_number TEXT, feet_from_curb INTEGER, n
o_standing_or_stopping_violation TEXT, hydrant_violation TEXT, double_parking_violation TEXT, pg_created_at T
IMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
ons_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
s_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 100000 rows into parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['summons_number', 'r
egistration_state', 'plate_type', 'issue_date', 'violation_code', 'vehicle_body_type', 'vehicle_make', 'issui
ng_agency', 'vehicle_expiration_date', 'violation_location', 'violation_precinct', 'issuer_precinct', 'issuer
_code', 'issuer_command', 'issuer_squad', 'violation_time', 'time_first_observed', 'violation_county', 'viola
tion_in_front_of_or_opposite', 'date_first_observed', 'law_section', 'sub_division', 'violation_legal_code',
'days_parking_in_effect', 'from_hours_in_effect', 'to_hours_in_effect', 'vehicle_color', 'unregistered_vehicl
e', 'vehicle_year', 'meter_number', 'feet_from_curb', 'no_standing_or_stopping_violation', 'hydrant_violatio
n', 'double_parking_violation']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['summons_number']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 100000 rows in parking_
violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 100000 rows
```

```
INFO - __main__ - Completed ingestion for parking_violations_issued
INFO - __main__ - Found 2 public tables
INFO - __main__ - Public tables dataframe:
               table_name
  parking_violation_codes
parking_violations_issued
```

We can confirm that the ingested data is clean again by running the same report.

In [8…

```
sql_query = """
SELECT
    parking_violation_codes.code,
    parking_violation_codes.definition,
    COUNT(parking_violations_issued.violation_code) AS ticket_count
FROM
    parking_violation_codes
JOIN
    parking_violations_issued ON
    parking_violation_codes.code = parking_violations_issued.violation_code
GROUP BY
    parking_violation_codes.code,
    parking_violation_codes.definition
ORDER BY
    ticket_count DESC
LIMIT 10;
"""

postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Out[8…

	code	definition	ticket_count
0	36	PHTO SCHOOL ZN SPEED VIOLATION	37064
1	21	NO PARKING-STREET CLEANING	12149
2	38	FAIL TO DSPLY MUNI METER RECPT	6482
3	14	NO STANDING-DAY/TIME LIMITS	5047
4	7	FAILURE TO STOP AT RED LIGHT	4055
5	20	NO PARKING-DAY/TIME LIMITS	3949
6	40	FIRE HYDRANT	3902
7	71	INSP. STICKER-EXPIRED/MISSING	3655
8	5	BUS LANE VIOLATION	3477
9	70	REG. STICKER-EXPIRED/MISSING	2595

Let's now implement CHECK constraints to prevent the bad data from being ingested. We will add the following constraints to the `parking_violation_codes` table.

To learn more about CHECK constraints, see the following PostgreSQL documentation:

https://www.postgresql.org/docs/current/ddl-constraints.html (https://www.postgresql.org/docs/current/ddl-constraints.html)

In [9…

```
sql_query = """
ALTER TABLE parking_violation_codes ADD CONSTRAINT code_check CHECK (code >= 0);
"""

postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Let's run the ingestion again via PostgresIngestionCSV().ingest() to see the results when we try to ingest the dirty data for the `parking_violation_codes` table.

In [1…

```
PostgresIngestionCSV().ingest(
    csv_path='data_platform/data/dirty_data/dirty_dof_parking_violation_codes.csv',
    schema_path='data_platform/data/schemas/parking_violation_codes.json'
)
```

```
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/dirty_data/dir
ty_dof_parking_violation_codes.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violation_codes.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['CODE', 'DEFINITION'] -> ['co
de', 'definition']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 1 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violation_codes with 4 column
s
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
on_codes:

            CREATE TABLE IF NOT EXISTS parking_violation_codes (
                code INTEGER PRIMARY KEY, definition TEXT, manhattan_96th_st_below INTEGER, all_other_areas I
NTEGER, pg_created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
on_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 1 rows into parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['code', 'definitio
n']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['code']
---------------------------------------------------------------------------
CheckViolation                            Traceback (most recent call last)
Cell In[10], line 1
----> 1 PostgresIngestionCSV().ingest(
      2     csv_path='data_platform/data/dirty_data/dirty_dof_parking_violation_codes.csv',
      3     schema_path='data_platform/data/schemas/parking_violation_codes.json'
      4 )

File /workspaces/linkedin-learning-dq-upstream/data_platform/ingestion/postgres_ingestion_csv.py:50, in Postg
resIngestionCSV.ingest(self, csv_path, schema_path)
     47 self.logger.info(f"Processing {cleaned_rows} rows for ingestion")
     49 self._create_table(schema)
---> 50 self._insert_data(df_col_cleaned, schema)
     52 self.logger.info(
     53     f"Successfully completed ingestion of {cleaned_rows} rows"
     54     )

File /workspaces/linkedin-learning-dq-upstream/data_platform/ingestion/postgres_ingestion_csv.py:220, in Post
gresIngestionCSV._insert_data(self, df, schema)
    215 values_list = [
    216     tuple(row) for row in df_ordered.itertuples(index=False)
    217 ]
    219 # Use executemany for bulk UPSERT (psycopg v3 optimized)
--> 220 cursor.executemany(upsert_query, values_list)
    221 connection.commit()
    223 self.logger.info(
    224     f"Successfully inserted/updated {row_count} rows "
    225     f"in {table_name}"
    226     )

File /usr/local/lib/python3.10/site-packages/psycopg/cursor.py:128, in Cursor.executemany(self, query, params
_seq, returning)
    124             self._conn.wait(
    125                 self._executemany_gen_no_pipeline(query, params_seq, returning)
    126             )
    127 except e._NO_TRACEBACK as ex:
--> 128     raise ex.with_traceback(None)

CheckViolation: new row for relation "parking_violation_codes" violates check constraint "code_check"
DETAIL:  Failing row contains (-99, TEST, null, null, 2025-09-27 05:43:45.334099, 2025-09-27 05:43:45.33409
9).
```

Amazing! We are now preventing bad data from being ingested for the `parking_violation_codes` table.

```
CheckViolation: new row for relation "parking_violation_codes" violates check constraint "code_chec
k"
DETAIL:  Failing row contains (-99, TEST, null, null, 2025-09-26 22:43:18.335114, 2025-09-26 22:43:
18.335114).
```

Now let's implement a similar check for the `parking_violations_issued` table.

In [1…

```
sql_query = """
ALTER TABLE parking_violations_issued
  ADD CONSTRAINT summons_number_is_ten_digits
  CHECK (summons_number::text ~ '^[0-9]{10}$');
"""

postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Let's run the ingestion again via PostgresIngestionCSV().ingest() to see the results when we try to ingest the dirty data for the `parking_violations_issued` table.

In [1…

```
PostgresIngestionCSV().ingest(
    csv_path='data_platform/data/dirty_data/dirty_parking_violations_issued.csv',
    schema_path='data_platform/data/schemas/parking_violations_issued.json'
)
```

```
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/dirty_data/dir
ty_parking_violations_issued.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violations_issued.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['Summons Number', 'Violation
Code'] -> ['summons_number', 'violation_code']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 10000 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violations_issued with 34 col
umns
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
ons_issued:

            CREATE TABLE IF NOT EXISTS parking_violations_issued (
                summons_number BIGINT PRIMARY KEY, registration_state TEXT, plate_type TEXT, issue_date DATE,
violation_code INTEGER, vehicle_body_type TEXT, vehicle_make TEXT, issuing_agency TEXT, vehicle_expiration_da
te TEXT, violation_location TEXT, violation_precinct INTEGER, issuer_precinct INTEGER, issuer_code INTEGER, i
ssuer_command TEXT, issuer_squad TEXT, violation_time TEXT, time_first_observed TEXT, violation_county TEXT,
violation_in_front_of_or_opposite TEXT, date_first_observed TEXT, law_section INTEGER, sub_division TEXT, vio
lation_legal_code TEXT, days_parking_in_effect TEXT, from_hours_in_effect TEXT, to_hours_in_effect TEXT, vehi
cle_color TEXT, unregistered_vehicle TEXT, vehicle_year INTEGER, meter_number TEXT, feet_from_curb INTEGER, n
o_standing_or_stopping_violation TEXT, hydrant_violation TEXT, double_parking_violation TEXT, pg_created_at T
IMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
ons_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
s_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 10000 rows into parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['summons_number', 'v
iolation_code']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['summons_number']
---------------------------------------------------------------------------
CheckViolation                            Traceback (most recent call last)
Cell In[12], line 1
----> 1 PostgresIngestionCSV().ingest(
      2     csv_path='data_platform/data/dirty_data/dirty_parking_violations_issued.csv',
      3     schema_path='data_platform/data/schemas/parking_violations_issued.json'
      4 )

File /workspaces/linkedin-learning-dq-upstream/data_platform/ingestion/postgres_ingestion_csv.py:50, in Postg
resIngestionCSV.ingest(self, csv_path, schema_path)
     47 self.logger.info(f"Processing {cleaned_rows} rows for ingestion")
     49 self._create_table(schema)
---> 50 self._insert_data(df_col_cleaned, schema)
     52 self.logger.info(
     53     f"Successfully completed ingestion of {cleaned_rows} rows"
     54     )

File /workspaces/linkedin-learning-dq-upstream/data_platform/ingestion/postgres_ingestion_csv.py:220, in Post
gresIngestionCSV._insert_data(self, df, schema)
    215 values_list = [
    216     tuple(row) for row in df_ordered.itertuples(index=False)
    217 ]
    219 # Use executemany for bulk UPSERT (psycopg v3 optimized)
--> 220 cursor.executemany(upsert_query, values_list)
    221 connection.commit()
    223 self.logger.info(
    224     f"Successfully inserted/updated {row_count} rows "
    225     f"in {table_name}"
    226     )

File /usr/local/lib/python3.10/site-packages/psycopg/cursor.py:128, in Cursor.executemany(self, query, params
_seq, returning)
    124             self._conn.wait(
    125                 self._executemany_gen_no_pipeline(query, params_seq, returning)
    126             )
    127 except e._NO_TRACEBACK as ex:
--> 128     raise ex.with_traceback(None)

CheckViolation: new row for relation "parking_violations_issued" violates check constraint "summons_number_is
_ten_digits"
DETAIL:  Failing row contains (999077846086, null, null, null, -99, null, null, null, null, null, null, null,
null, null, null, null, null, null, null, null, null, null, null, null, null, null, null, null, null, null, n
ull, null, null, null, 2025-09-27 05:43:48.276551, 2025-09-27 05:43:48.276551).
```

Amazing! We are now preventing bad data from being ingested for the `parking_violations_issued` table as well.

```
CheckViolation: new row for relation "parking_violation_codes" violates check constraint "code_chec
k"
DETAIL:  Failing row contains (-99, TEST, null, null, 2025-09-26 22:43:18.335114, 2025-09-26 22:43:
18.335114).
```

1.4 - Ingestion Code Exploration

Take a moment to explore the ingestion code. One thing to note is that this pipleine is using custom ingestion code, but in a more robust data platform, you would often use a "data migration tool" so that you can version control your updates to the database.

Chapter 2 - Transactions

2.1 Database Transaction Pitfalls

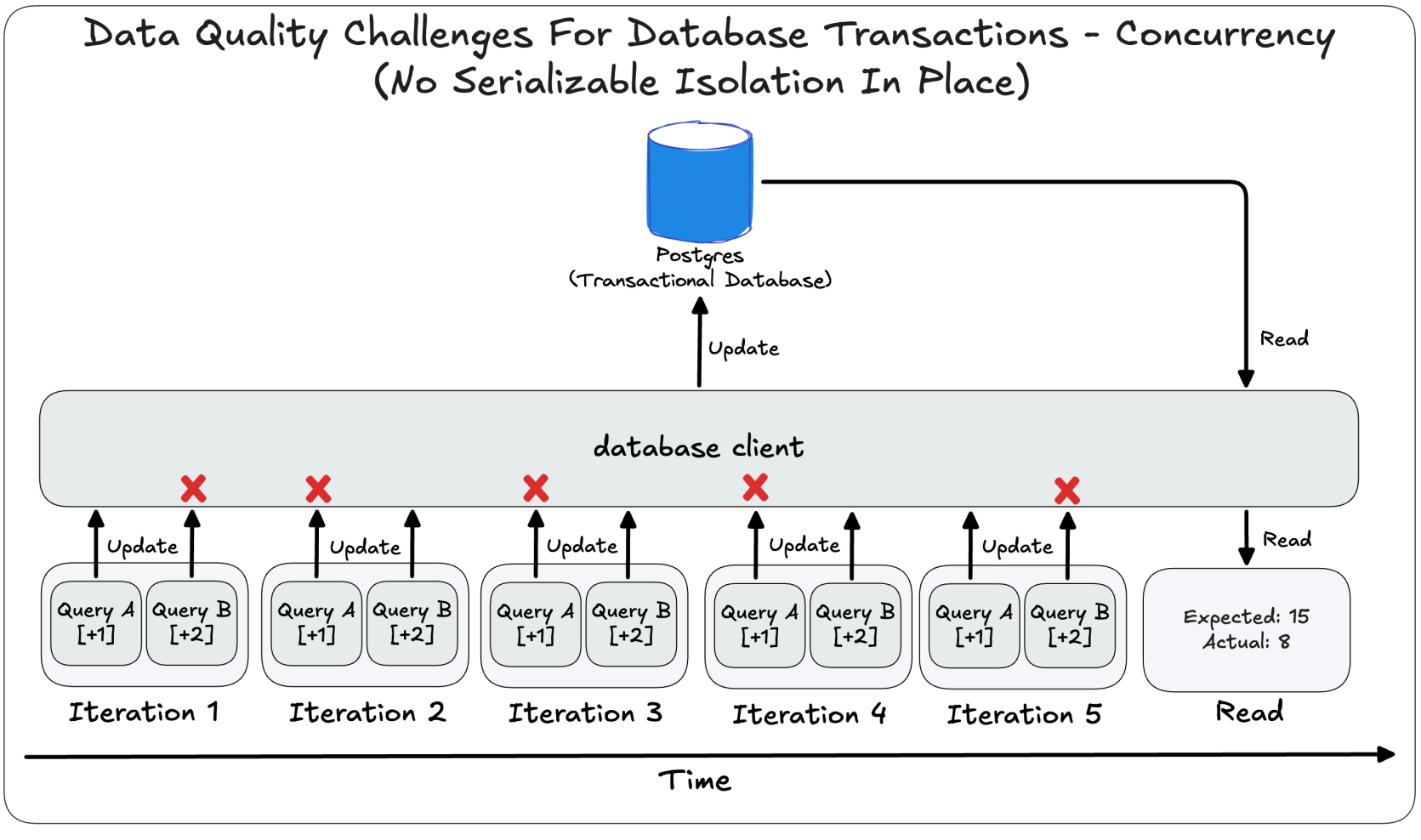
When it comes to databases, data quality goes well beyond the schema and data itself. Specifically in production environments, we need to expect the "unexpected" and build protections against it. Now, this topic can be an entire course on its own, and I highly recommend reading *Chapter 8: Transactions* in the book *Designing Data-Intensive Applications, 2nd Edition* if you want a more in-depth look at this topic (I based this section on the book).

With that said, for a free resource that the above book references, I again recommend PostgresSQL's official documentaton:

- PostgreSQL Documentation - Chapter 13. Concurrency Control (https://www.postgresql.org/docs/current/mvcc.html)
- PostgreSQL Documentation - 13.2. Transaction Isolation (https://www.postgresql.org/docs/current/transaction-iso.html)
- PostgreSQL Documentation - SET TRANSACTION (https://www.postgresql.org/docs/current/sql-set-transaction.html)

A situation that we will focus on in this chapter is the "lost update" problem, which is a specific manifestation of a serialization anomaly. This occurs when two transactions read the same data, make changes, and then one of the changes is lost, resulting in an outcome that would be impossible if the transactions were executed one at a time in any order.

The following diagram shows the problem:



This is super theory heavy, furthermore, it's hard to replicate this organically as we are not working with a multi-user environment. So instead, we will simulate the problem using the `TransactionSimulator` class that we initiated in the setup of the sandbox environment.

2.2 - Simulating the Lost Update Problem

We will first simulate the problem without any defined isolation levels. Given that the failures are random, we are likely to get different results each time we run the simulation (this is expected and the exact problem we are trying to highlight).

> [!NOTE] By default, PostgreSQL uses the `READ COMMITTED` isolation level in the background of all queries.

In [1…

```
simulator.simulate(increment_a=1, increment_b=2, iterations=5)
```

```
INFO - data_platform.transactions.concurrency_simulator - Starting simulation with 5 iterations, increments 1
& 2
INFO - data_platform.transactions.concurrency_simulator - Setting up transaction demo table
INFO - data_platform.transactions.concurrency_simulator - Resetting counter to 0
INFO - data_platform.transactions.concurrency_simulator - Counter reset to 0
INFO - data_platform.transactions.concurrency_simulator - Demo table setup complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 1/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 2
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 1
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 2
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 1
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 1
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 1: 1
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 1:
 id  counter
  1        1
INFO - data_platform.transactions.concurrency_simulator - Iteration 1 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 2/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 3
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 2
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 3
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 2
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 2
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 2: 2
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 2:
 id  counter
  1        2
INFO - data_platform.transactions.concurrency_simulator - Iteration 2 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 3/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 4
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 3
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 3
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 4
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 4
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 3: 4
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 3:
 id  counter
  1        4
INFO - data_platform.transactions.concurrency_simulator - Iteration 3 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 4/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 6
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 5
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 6
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 5
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 5
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 4: 5
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 4:
 id  counter
  1        5
INFO - data_platform.transactions.concurrency_simulator - Iteration 4 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 5/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 7
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 6
```

```
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 7
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 6
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 6
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 5: 6
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 5:
 id  counter
  1        6
INFO - data_platform.transactions.concurrency_simulator - Iteration 5 complete
```

Now let's run the simulation again with the same settings and see how the results change. You may need to re-run the cell due to the small chance that the two random results are the same.

In [1…

```
simulator.simulate(increment_a=1, increment_b=2, iterations=5)
```

```
INFO - data_platform.transactions.concurrency_simulator - Starting simulation with 5 iterations, increments 1
& 2
INFO - data_platform.transactions.concurrency_simulator - Setting up transaction demo table
INFO - data_platform.transactions.concurrency_simulator - Resetting counter to 0
INFO - data_platform.transactions.concurrency_simulator - Counter reset to 0
INFO - data_platform.transactions.concurrency_simulator - Demo table setup complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 1/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 1
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 2
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 1
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 2
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 2
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 1: 2
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 1:
 id  counter
  1        2
INFO - data_platform.transactions.concurrency_simulator - Iteration 1 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 2/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 3
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 4
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 3
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 4
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 4
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 2: 4
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 2:
 id  counter
  1        4
INFO - data_platform.transactions.concurrency_simulator - Iteration 2 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 3/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 6
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 5
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 6
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 5
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 5
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 3: 5
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 3:
 id  counter
  1        5
INFO - data_platform.transactions.concurrency_simulator - Iteration 3 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 4/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 6
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 7
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 6
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 7
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 7
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 4: 7
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 4:
 id  counter
  1        7
INFO - data_platform.transactions.concurrency_simulator - Iteration 4 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 5/5 ---
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 8
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 9
```

```
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 9
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 8
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 8
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 5: 8
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 5:
 id  counter
  1        8
INFO - data_platform.transactions.concurrency_simulator - Iteration 5 complete
```

2.3 - Preventing Concurrency Issues With Isolation

You can now see that the same set of transactions are resulting in different numbers! This is how the database systems themselves result in data quality issues, and why we need to be proactive about it.

Now let's see how the problem is solved by setting the isolation level to `SERIALIZABLE`.

In [1…

```
simulator.simulate(increment_a=1, increment_b=2, iterations=5, isolation_level="SERIALIZABLE")
```

```
INFO - data_platform.transactions.concurrency_simulator - Starting simulation with 5 iterations, increments 1
& 2, isolation: SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Setting up transaction demo table
INFO - data_platform.transactions.concurrency_simulator - Resetting counter to 0
INFO - data_platform.transactions.concurrency_simulator - Counter reset to 0
INFO - data_platform.transactions.concurrency_simulator - Demo table setup complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 1/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 2
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 1
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 2
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 3
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 3
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 3
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 1: 3
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 1:
 id  counter
  1        3
INFO - data_platform.transactions.concurrency_simulator - Iteration 1 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 2/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 5
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 4
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 5
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 6
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 6
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 6
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 2: 6
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 2:
 id  counter
  1        6
INFO - data_platform.transactions.concurrency_simulator - Iteration 2 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 3/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 7
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 8
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 8
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 9
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 9
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 9
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 3: 9
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 3:
 id  counter
  1        9
INFO - data_platform.transactions.concurrency_simulator - Iteration 3 complete
```

```
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 4/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 11
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 10
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 11
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 12
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 12
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 12
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 4: 12
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 4:
 id  counter
  1       12
INFO - data_platform.transactions.concurrency_simulator - Iteration 4 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 5/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 14
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 13
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 14
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 15
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 15
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 15
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 5: 15
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 5:
 id  counter
  1       15
INFO - data_platform.transactions.concurrency_simulator - Iteration 5 complete
```

Our result is now the expected value of 15! But let's run it again to confirm that the result is always the same.

In [1…

```
simulator.simulate(increment_a=1, increment_b=2, iterations=5, isolation_level="SERIALIZABLE")
```

```
INFO - data_platform.transactions.concurrency_simulator - Starting simulation with 5 iterations, increments 1
& 2, isolation: SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Setting up transaction demo table
INFO - data_platform.transactions.concurrency_simulator - Resetting counter to 0
INFO - data_platform.transactions.concurrency_simulator - Counter reset to 0
INFO - data_platform.transactions.concurrency_simulator - Demo table setup complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 1/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 1
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 2
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 1
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 3
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 3
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 3
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 1: 3
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 1:
 id  counter
  1        3
INFO - data_platform.transactions.concurrency_simulator - Iteration 1 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 2/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 4
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 5
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 4
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 6
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 6
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 6
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 2: 6
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 2:
 id  counter
  1        6
INFO - data_platform.transactions.concurrency_simulator - Iteration 2 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 3/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 7
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 8
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 7
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 9
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 9
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 9
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 3: 9
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 3:
 id  counter
  1        9
INFO - data_platform.transactions.concurrency_simulator - Iteration 3 complete
```

```
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 4/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 11
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 10
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 11
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 12
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 12
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 12
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 4: 12
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 4:
 id  counter
  1       12
INFO - data_platform.transactions.concurrency_simulator - Iteration 4 complete
INFO - data_platform.transactions.concurrency_simulator - --- Iteration 5/5 ---
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Executing: SET SESSION CHARACTERISTICS AS TRANSACTI
ON ISOLATION LEVEL SERIALIZABLE
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Waiting at barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Passed barrier
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 14
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 13
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 14
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serialize access d
ue to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
INFO - data_platform.transactions.concurrency_simulator - Calculated new value: 15
INFO - data_platform.transactions.concurrency_simulator - Sleeping for 1.0 seconds
INFO - data_platform.transactions.concurrency_simulator - Updated counter to 15
DEBUG - data_platform.transactions.concurrency_simulator - Executing: SELECT counter FROM transaction_simulat
ion WHERE id = 1
DEBUG - data_platform.transactions.concurrency_simulator - Current counter value: 15
INFO - data_platform.transactions.concurrency_simulator - Counter after iteration 5: 15
INFO - data_platform.transactions.concurrency_simulator -
transaction_simulation after iteration 5:
 id  counter
  1       15
INFO - data_platform.transactions.concurrency_simulator - Iteration 5 complete
```

They are the same! I want you to pay special attention to the logs of the simulation. You can see that the transactions are now being executed in order and the result is always the same.

Specifically check out the logs:

```
WARNING - data_platform.transactions.concurrency_simulator - Transaction failed: could not serializ
e access due to concurrent update
INFO - data_platform.transactions.concurrency_simulator - Retry attempt 1/4
```

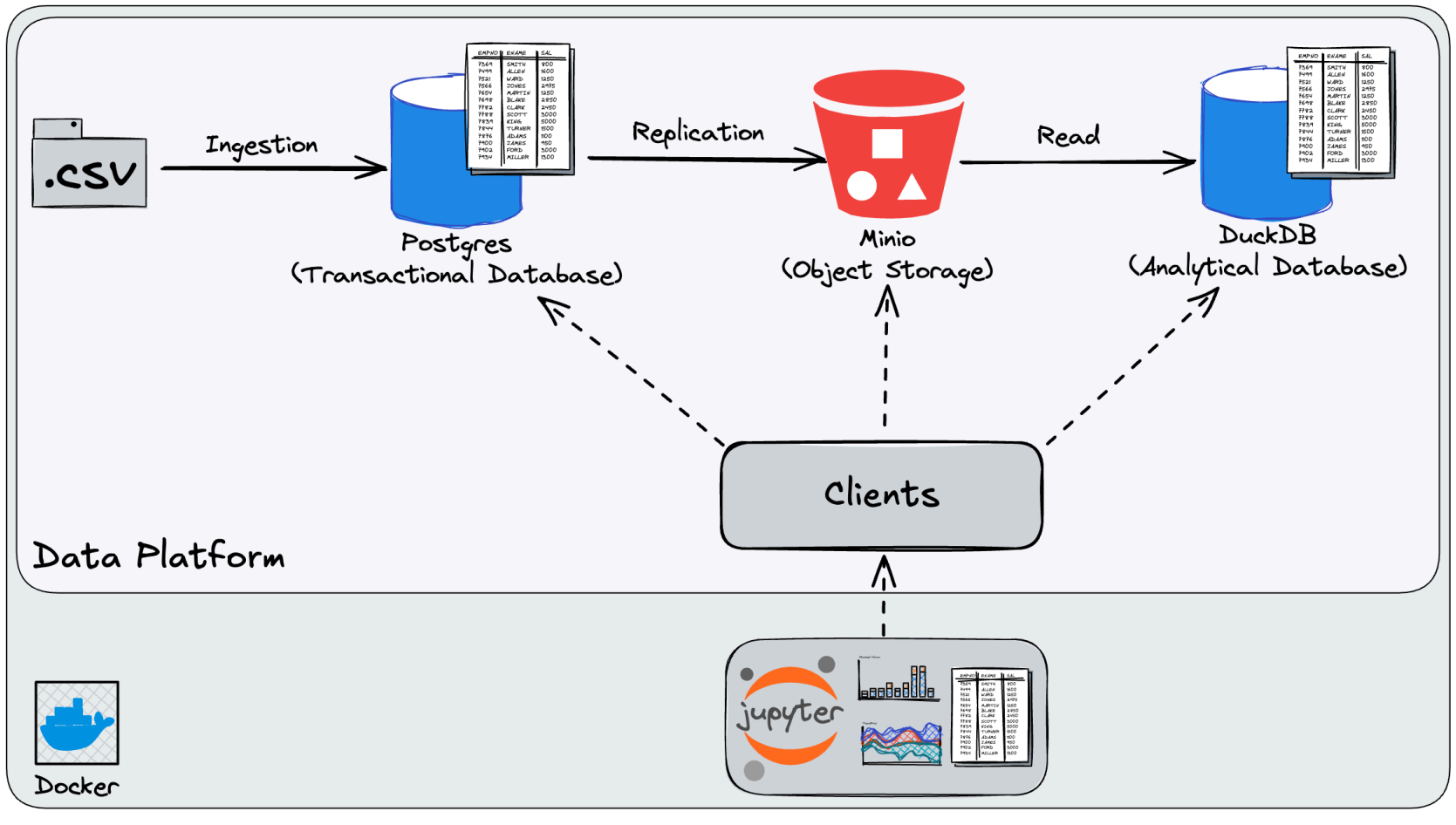
We can see that the transactions are now being executed in order and will retry a failed transaction up to 4 times before giving up.

2.4 - Ingestion Code Exploration

Take a moment to explore the transactions code. One thing to note is that this is only one type of concurrency problem that can occur, but this approach of isolation at the serializable level is the most conservative. The tradeoff is that it's much slower, so you need to be mindful of the performance need of your business use case.

Chapter 3 - Replication

3.1 - A Brief Overview of Data Lakehouses



Let's first clean up the PostgreSQL database to start fresh.

In [1…

```
sql_query = """
DROP TABLE IF EXISTS parking_violations_issued CASCADE;
DROP TABLE IF EXISTS parking_violation_codes CASCADE;
DROP TABLE IF EXISTS transaction_simulation CASCADE;
"""
postgres.execute_query(sql_query, return_pd_dataframe=True)
```

In [1…

```
query = """
SELECT
    table_name
FROM
    information_schema.tables
WHERE
    table_schema = 'public'
ORDER BY
    table_name;
"""
postgres.execute_query(query, return_pd_dataframe=True)
```

Out[1…

table_name

We can now run the ingestion and etl pipelines again.

In [1…

```
!python3 data_platform/scripts/run_ingestion.py
```

```
INFO - __main__ - Starting ingestion for parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/clean_data/dof
_parking_violation_codes.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violation_codes.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['CODE', 'DEFINITION', 'Manhat
tan\xa0 96th St. & below', 'All Other Areas'] -> ['code', 'definition', 'manhattan_96th_st_below', 'all_other
_areas']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 97 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violation_codes with 4 column
s
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
on_codes:

            CREATE TABLE IF NOT EXISTS parking_violation_codes (
                code INTEGER PRIMARY KEY, definition TEXT, manhattan_96th_st_below INTEGER, all_other_areas I
NTEGER, pg_created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
on_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 97 rows into parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['code', 'definitio
n', 'manhattan_96th_st_below', 'all_other_areas']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['code']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 97 rows in parking_viol
ation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 97 rows
INFO - __main__ - Completed ingestion for parking_violation_codes
INFO - __main__ - Starting ingestion for parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/clean_data/par
king_violations_issued_fiscal_year_2023_sample.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violations_issued.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['Summons Number', 'Registrati
on State', 'Plate Type', 'Issue Date', 'Violation Code', 'Vehicle Body Type', 'Vehicle Make', 'Issuing Agenc
y', 'Vehicle Expiration Date', 'Violation Location', 'Violation Precinct', 'Issuer Precinct', 'Issuer Code',
'Issuer Command', 'Issuer Squad', 'Violation Time', 'Time First Observed', 'Violation County', 'Violation In
Front Of Or Opposite', 'Date First Observed', 'Law Section', 'Sub Division', 'Violation Legal Code', 'Days Pa
rking In Effect    ', 'From Hours In Effect', 'To Hours In Effect', 'Vehicle Color', 'Unregistered Vehicle?',
'Vehicle Year', 'Meter Number', 'Feet From Curb', 'No Standing or Stopping Violation', 'Hydrant Violation',
'Double Parking Violation'] -> ['summons_number', 'registration_state', 'plate_type', 'issue_date', 'violatio
n_code', 'vehicle_body_type', 'vehicle_make', 'issuing_agency', 'vehicle_expiration_date', 'violation_locatio
n', 'violation_precinct', 'issuer_precinct', 'issuer_code', 'issuer_command', 'issuer_squad', 'violation_tim
e', 'time_first_observed', 'violation_county', 'violation_in_front_of_or_opposite', 'date_first_observed', 'l
aw_section', 'sub_division', 'violation_legal_code', 'days_parking_in_effect', 'from_hours_in_effect', 'to_ho
urs_in_effect', 'vehicle_color', 'unregistered_vehicle', 'vehicle_year', 'meter_number', 'feet_from_curb', 'n
o_standing_or_stopping_violation', 'hydrant_violation', 'double_parking_violation']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 100000 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violations_issued with 34 col
umns
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
ons_issued:

            CREATE TABLE IF NOT EXISTS parking_violations_issued (
                summons_number BIGINT PRIMARY KEY, registration_state TEXT, plate_type TEXT, issue_date DATE,
violation_code INTEGER, vehicle_body_type TEXT, vehicle_make TEXT, issuing_agency TEXT, vehicle_expiration_da
te TEXT, violation_location TEXT, violation_precinct INTEGER, issuer_precinct INTEGER, issuer_code INTEGER, i
ssuer_command TEXT, issuer_squad TEXT, violation_time TEXT, time_first_observed TEXT, violation_county TEXT,
violation_in_front_of_or_opposite TEXT, date_first_observed TEXT, law_section INTEGER, sub_division TEXT, vio
lation_legal_code TEXT, days_parking_in_effect TEXT, from_hours_in_effect TEXT, to_hours_in_effect TEXT, vehi
cle_color TEXT, unregistered_vehicle TEXT, vehicle_year INTEGER, meter_number TEXT, feet_from_curb INTEGER, n
o_standing_or_stopping_violation TEXT, hydrant_violation TEXT, double_parking_violation TEXT, pg_created_at T
IMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
ons_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
s_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 100000 rows into parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['summons_number', 'r
egistration_state', 'plate_type', 'issue_date', 'violation_code', 'vehicle_body_type', 'vehicle_make', 'issui
ng_agency', 'vehicle_expiration_date', 'violation_location', 'violation_precinct', 'issuer_precinct', 'issuer
_code', 'issuer_command', 'issuer_squad', 'violation_time', 'time_first_observed', 'violation_county', 'viola
tion_in_front_of_or_opposite', 'date_first_observed', 'law_section', 'sub_division', 'violation_legal_code',
'days_parking_in_effect', 'from_hours_in_effect', 'to_hours_in_effect', 'vehicle_color', 'unregistered_vehicl
e', 'vehicle_year', 'meter_number', 'feet_from_curb', 'no_standing_or_stopping_violation', 'hydrant_violatio
n', 'double_parking_violation']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['summons_number']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 100000 rows in parking_
violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 100000 rows
```

```
INFO - __main__ - Completed ingestion for parking_violations_issued
INFO - __main__ - Found 2 public tables
INFO - __main__ - Public tables dataframe:
               table_name
parking_violations_issued
  parking_violation_codes
```

In [2…

```
!python3 data_platform/scripts/run_etl.py
```

```
INFO - __main__ - Starting ETL: Postgres→MinIO (bucket=raw-data)
INFO - data_platform.etl.batch_postgres_minio_etl - Starting ETL pipeline for bucket: raw-data
INFO - data_platform.etl.batch_postgres_minio_etl - Querying PostgreSQL for public tables
INFO - data_platform.etl.batch_postgres_minio_etl - Found 2 tables to process: ['parking_violations_issued',
'parking_violation_codes']
INFO - data_platform.etl.batch_postgres_minio_etl - Processing table: parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 100000 rows from parking_violation
s_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Transforming 100000 records to Avro format for table: par
king_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violations_issu
ed
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violations_issued.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 36 fields for parking_violati
ons_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully transformed 100000 records toAvro format
INFO - data_platform.etl.batch_postgres_minio_etl - Loading 100000 records to MinIO bucket: raw-data
DEBUG - data_platform.etl.batch_postgres_minio_etl - Writing Avro file: /tmp/parking_violations_issued_202509
27_054457.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Uploading to MinIO: postgres-exports/parking_violations_
issued/2025/09/27/parking_violations_issued_20250927_054457.avro
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "HEAD /raw-data HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "POST /raw-data/postgres-exports/parking_violations_issue
d/2025/09/27/parking_violations_issued_20250927_054457.avro?uploads= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - Starting new HTTP connection (2): minio:9000
DEBUG - urllib3.connectionpool - Starting new HTTP connection (3): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /raw-data/postgres-exports/parking_violations_issued/
2025/09/27/parking_violations_issued_20250927_054457.avro?partNumber=3&uploadId=ZjExNzMzMWUtYjU2NC00NmU0LTg0Z
TAtNWU4MjkxMjVmMDk0LjczNWRlOTMwLWUxY2QtNGM1Zi1iMmI1LTEzNWUwNzM3OGQ1OHgxNzU4OTUxODk3NzkwODE5MDg1 HTTP/1.1" 200
0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /raw-data/postgres-exports/parking_violations_issued/
2025/09/27/parking_violations_issued_20250927_054457.avro?partNumber=1&uploadId=ZjExNzMzMWUtYjU2NC00NmU0LTg0Z
TAtNWU4MjkxMjVmMDk0LjczNWRlOTMwLWUxY2QtNGM1Zi1iMmI1LTEzNWUwNzM3OGQ1OHgxNzU4OTUxODk3NzkwODE5MDg1 HTTP/1.1" 200
0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /raw-data/postgres-exports/parking_violations_issued/
2025/09/27/parking_violations_issued_20250927_054457.avro?partNumber=2&uploadId=ZjExNzMzMWUtYjU2NC00NmU0LTg0Z
TAtNWU4MjkxMjVmMDk0LjczNWRlOTMwLWUxY2QtNGM1Zi1iMmI1LTEzNWUwNzM3OGQ1OHgxNzU4OTUxODk3NzkwODE5MDg1 HTTP/1.1" 200
0
DEBUG - urllib3.connectionpool - http://minio:9000 "POST /raw-data/postgres-exports/parking_violations_issue
d/2025/09/27/parking_violations_issued_20250927_054457.avro?uploadId=ZjExNzMzMWUtYjU2NC00NmU0LTg0ZTAtNWU4Mjkx
MjVmMDk0LjczNWRlOTMwLWUxY2QtNGM1Zi1iMmI1LTEzNWUwNzM3OGQ1OHgxNzU4OTUxODk3NzkwODE5MDg1 HTTP/1.1" 200 0
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully uploaded 100000 records to postgres-exports/
parking_violations_issued/2025/09/27/parking_violations_issued_20250927_054457.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Cleaned up temporary file: /tmp/parking_violations_issue
d_20250927_054457.avro
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully processed table: parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Processing table: parking_violation_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: parking_violation_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 97 rows from parking_violation_cod
es
INFO - data_platform.etl.batch_postgres_minio_etl - Transforming 97 records to Avro format for table: parking
_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violation_codes.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 6 fields for parking_violatio
n_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully transformed 97 records toAvro format
INFO - data_platform.etl.batch_postgres_minio_etl - Loading 97 records to MinIO bucket: raw-data
DEBUG - data_platform.etl.batch_postgres_minio_etl - Writing Avro file: /tmp/parking_violation_codes_20250927
_054457.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Uploading to MinIO: postgres-exports/parking_violation_c
odes/2025/09/27/parking_violation_codes_20250927_054457.avro
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "HEAD /raw-data HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /raw-data/postgres-exports/parking_violation_codes/20
25/09/27/parking_violation_codes_20250927_054457.avro HTTP/1.1" 200 0
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully uploaded 97 records to postgres-exports/park
ing_violation_codes/2025/09/27/parking_violation_codes_20250927_054457.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Cleaned up temporary file: /tmp/parking_violation_codes_
20250927_054457.avro
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully processed table: parking_violation_codes
INFO - data_platform.etl.batch_postgres_minio_etl - ETL pipeline completed successfully
INFO - __main__ - Completed ETL: Postgres→MinIO
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?delimiter=&encoding-type=url&list-type=2&ma
x-keys=1000&prefix= HTTP/1.1" 200 0
INFO - __main__ - minio:raw-data/postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_
20250927_053734.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_
20250927_054008.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_
20250927_054457.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violations_issued/2025/09/27/parking_violations_iss
ued_20250927_053735.avro
```

```
INFO - __main__ - minio:raw-data/postgres-exports/parking_violations_issued/2025/09/27/parking_violations_iss
ued_20250927_054008.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violations_issued/2025/09/27/parking_violations_iss
ued_20250927_054457.avro
INFO - data_platform.etl.batch_minio_duckdb_elt - Extracting latest Avro for table 'parking_violation_codes'
from 'postgres-exports'
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?delimiter=&encoding-type=url&list-type=2&ma
x-keys=1000&prefix= HTTP/1.1" 200 0
INFO - data_platform.etl.batch_minio_duckdb_elt - Found 3 candidates, latest: postgres-exports/parking_violat
ion_codes/2025/09/27/parking_violation_codes_20250927_054457.avro
INFO - data_platform.etl.batch_minio_duckdb_elt - Loading 's3://raw-data/postgres-exports/parking_violation_c
odes/2025/09/27/parking_violation_codes_20250927_054457.avro' into staging.parking_violation_codes
DEBUG - data_platform.etl.batch_minio_duckdb_elt - Creating staging schema
INFO - data_platform.etl.batch_minio_duckdb_elt - Creating table staging.parking_violation_codes
INFO - data_platform.etl.batch_minio_duckdb_elt - Successfully loaded 97 rows into staging.parking_violation_
codes
INFO - __main__ - Completed ELT for table=parking_violation_codes
INFO - data_platform.etl.batch_minio_duckdb_elt - Extracting latest Avro for table 'parking_violations_issue
d' from 'postgres-exports'
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?delimiter=&encoding-type=url&list-type=2&ma
x-keys=1000&prefix= HTTP/1.1" 200 0
INFO - data_platform.etl.batch_minio_duckdb_elt - Found 3 candidates, latest: postgres-exports/parking_violat
ions_issued/2025/09/27/parking_violations_issued_20250927_054457.avro
INFO - data_platform.etl.batch_minio_duckdb_elt - Loading 's3://raw-data/postgres-exports/parking_violations_
issued/2025/09/27/parking_violations_issued_20250927_054457.avro' into staging.parking_violations_issued
DEBUG - data_platform.etl.batch_minio_duckdb_elt - Creating staging schema
INFO - data_platform.etl.batch_minio_duckdb_elt - Creating table staging.parking_violations_issued
INFO - data_platform.etl.batch_minio_duckdb_elt - Successfully loaded 100000 rows into staging.parking_violat
ions_issued
INFO - __main__ - Completed ELT for table=parking_violations_issued
INFO - __main__ - DuckDB staging tables:
    database   schema                       name
0  lakehouse  staging    parking_violation_codes
1  lakehouse  staging  parking_violations_issued
```

To confirm that the data files are in the MinIO bucket, we can run the following cell code:

In [2…

```
objects = minio.list_objects("raw-data")
print("Files in raw-data bucket:")
for obj in sorted(objects):
    print(f"  {obj}")
```

```
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?delimiter=&encoding-type=url&list-type=2&ma
x-keys=1000&prefix= HTTP/1.1" 200 0
Files in raw-data bucket:
  postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_20250927_053734.avro
  postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_20250927_054008.avro
  postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_20250927_054457.avro
  postgres-exports/parking_violations_issued/2025/09/27/parking_violations_issued_20250927_053735.avro
  postgres-exports/parking_violations_issued/2025/09/27/parking_violations_issued_20250927_054008.avro
  postgres-exports/parking_violations_issued/2025/09/27/parking_violations_issued_20250927_054457.avro
```

Pay special attention to how we have multiple files for the same data assets! This allows us to have a snapshot of the data at the time of replication. When DuckDB reads the data, it will read the latest file for each data asset.

Now let's run the same report, but in DuckDB instead of PostgreSQL, as before to confirm that the data is consistent.

In [2…

```
query = """
SELECT
    parking_violation_codes.code,
    parking_violation_codes.definition,
    COUNT(parking_violations_issued.violation_code) AS ticket_count
FROM
    staging.parking_violation_codes
JOIN
    staging.parking_violations_issued ON
    parking_violation_codes.code = parking_violations_issued.violation_code
GROUP BY
    parking_violation_codes.code,
    parking_violation_codes.definition
ORDER BY
    ticket_count DESC
LIMIT 10;
"""
with DuckdbClient(db_path="data_platform/lakehouse/lakehouse.db") as duckdb:
    display(duckdb.execute_query(query, return_pd_dataframe=True))
```

	code	definition	ticket_count
0	36	PHTO SCHOOL ZN SPEED VIOLATION	37064
1	21	NO PARKING-STREET CLEANING	12149
2	38	FAIL TO DSPLY MUNI METER RECPT	6482
3	14	NO STANDING-DAY/TIME LIMITS	5047
4	7	FAILURE TO STOP AT RED LIGHT	4055
5	20	NO PARKING-DAY/TIME LIMITS	3949
6	40	FIRE HYDRANT	3902
7	71	INSP. STICKER-EXPIRED/MISSING	3655
8	5	BUS LANE VIOLATION	3477
9	70	REG. STICKER-EXPIRED/MISSING	2595

Confirmed that the data is consistent!

3.2 - Intentionally Breaking Our Replication Pipeline

Now let's break the pipeline by adding dirty data into the data platform. Note that our previous ingestion checks in PostgreSQL are no longer present since we dropped the tables to start fresh. This is intentional so we can persist the dirty data to this stage of the pipeline.

Since it's dirty data, we will need to run the manual ingestion process again.

In [2…

```
PostgresIngestionCSV().ingest(
    csv_path='data_platform/data/dirty_data/dirty_parking_violations_issued.csv',
    schema_path='data_platform/data/schemas/parking_violations_issued.json'
)

PostgresIngestionCSV().ingest(
    csv_path='data_platform/data/dirty_data/dirty_dof_parking_violation_codes.csv',
    schema_path='data_platform/data/schemas/parking_violation_codes.json'
)
```

```
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/dirty_data/dir
ty_parking_violations_issued.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violations_issued.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['Summons Number', 'Violation
Code'] -> ['summons_number', 'violation_code']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 10000 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violations_issued with 34 col
umns
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
ons_issued:

            CREATE TABLE IF NOT EXISTS parking_violations_issued (
                summons_number BIGINT PRIMARY KEY, registration_state TEXT, plate_type TEXT, issue_date DATE,
violation_code INTEGER, vehicle_body_type TEXT, vehicle_make TEXT, issuing_agency TEXT, vehicle_expiration_da
te TEXT, violation_location TEXT, violation_precinct INTEGER, issuer_precinct INTEGER, issuer_code INTEGER, i
ssuer_command TEXT, issuer_squad TEXT, violation_time TEXT, time_first_observed TEXT, violation_county TEXT,
violation_in_front_of_or_opposite TEXT, date_first_observed TEXT, law_section INTEGER, sub_division TEXT, vio
lation_legal_code TEXT, days_parking_in_effect TEXT, from_hours_in_effect TEXT, to_hours_in_effect TEXT, vehi
cle_color TEXT, unregistered_vehicle TEXT, vehicle_year INTEGER, meter_number TEXT, feet_from_curb INTEGER, n
o_standing_or_stopping_violation TEXT, hydrant_violation TEXT, double_parking_violation TEXT, pg_created_at T
IMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
ons_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
s_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 10000 rows into parking_violations_issued
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['summons_number', 'v
iolation_code']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['summons_number']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 10000 rows in parking_v
iolations_issued
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 10000 rows
INFO - data_platform.ingestion.postgres_ingestion_csv - Starting ingestion: data_platform/data/dirty_data/dir
ty_dof_parking_violation_codes.csv -> PostgreSQL
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Loading schema from: data_platform/data/schemas/park
ing_violation_codes.json
INFO - data_platform.ingestion.postgres_ingestion_csv - Loaded schema for table: parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Cleaned column names: ['CODE', 'DEFINITION'] -> ['co
de', 'definition']
INFO - data_platform.ingestion.postgres_ingestion_csv - Processing 1 rows for ingestion
INFO - data_platform.ingestion.postgres_ingestion_csv - Creating table structure and triggers for: parking_vi
olation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating table parking_violation_codes with 4 column
s
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Generated SQL create table query for parking_violati
on_codes:

            CREATE TABLE IF NOT EXISTS parking_violation_codes (
                code INTEGER PRIMARY KEY, definition TEXT, manhattan_96th_st_below INTEGER, all_other_areas I
NTEGER, pg_created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, pg_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );

DEBUG - data_platform.ingestion.postgres_ingestion_csv - Setting up update trigger for table: parking_violati
on_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Creating update trigger for table: parking_violation
_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Table setup completed for: parking_violation_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Inserting 1 rows into parking_violation_codes
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Matched columns for insertion: ['code', 'definitio
n']
DEBUG - data_platform.ingestion.postgres_ingestion_csv - Primary keys: ['code']
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully inserted/updated 1 rows in parking_viola
tion_codes
INFO - data_platform.ingestion.postgres_ingestion_csv - Successfully completed ingestion of 1 rows
```

Now let's run the same report again and see the results and confirm the dirty data is present.

In [2…
```
sql_query = """
SELECT
    parking_violation_codes.code,
    parking_violation_codes.definition,
    COUNT(parking_violations_issued.violation_code) AS ticket_count
FROM
    parking_violation_codes
JOIN
    parking_violations_issued ON
    parking_violation_codes.code = parking_violations_issued.violation_code
GROUP BY
    parking_violation_codes.code,
    parking_violation_codes.definition
ORDER BY
    ticket_count DESC
LIMIT 10;
"""

postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Out[2…

	code	definition	ticket_count
0	36	PHTO SCHOOL ZN SPEED VIOLATION	37064
1	21	NO PARKING-STREET CLEANING	12149
2	-99	TEST	10000
3	38	FAIL TO DSPLY MUNI METER RECPT	6482
4	14	NO STANDING-DAY/TIME LIMITS	5047
5	7	FAILURE TO STOP AT RED LIGHT	4055
6	20	NO PARKING-DAY/TIME LIMITS	3949
7	40	FIRE HYDRANT	3902
8	71	INSP. STICKER-EXPIRED/MISSING	3655
9	5	BUS LANE VIOLATION	3477

Let's now run the ETL pipeline again to see the results.

In [2…
```
!python3 data_platform/scripts/run_etl.py
```

```
INFO - __main__ - Starting ETL: Postgres→MinIO (bucket=raw-data)
INFO - data_platform.etl.batch_postgres_minio_etl - Starting ETL pipeline for bucket: raw-data
INFO - data_platform.etl.batch_postgres_minio_etl - Querying PostgreSQL for public tables
INFO - data_platform.etl.batch_postgres_minio_etl - Found 2 tables to process: ['parking_violations_issued',
'parking_violation_codes']
INFO - data_platform.etl.batch_postgres_minio_etl - Processing table: parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 110000 rows from parking_violation
s_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Transforming 110000 records to Avro format for table: par
king_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violations_issu
ed
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violations_issued.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 36 fields for parking_violati
ons_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully transformed 110000 records toAvro format
INFO - data_platform.etl.batch_postgres_minio_etl - Loading 110000 records to MinIO bucket: raw-data
DEBUG - data_platform.etl.batch_postgres_minio_etl - Writing Avro file: /tmp/parking_violations_issued_202509
27_054508.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Cleaned up temporary file: /tmp/parking_violations_issue
d_20250927_054508.avro
ERROR - data_platform.etl.batch_postgres_minio_etl - Failed to process table parking_violations_issued: must
be string on field registration_state
ERROR - __main__ - ETL failed
Traceback (most recent call last):
  File "fastavro/_write.pyx", line 130, in fastavro._write.write_utf8
AttributeError: 'NoneType' object has no attribute 'encode'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "fastavro/_write.pyx", line 437, in fastavro._write.write_data
  File "fastavro/_write.pyx", line 132, in fastavro._write.write_utf8
TypeError: must be string

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "/workspaces/linkedin-learning-dq-upstream/data_platform/scripts/run_etl.py", line 40, in run_postgres
_to_minio_etl
    etl.run_etl(bucket_name=MINIO_BUCKET)
  File "/workspaces/linkedin-learning-dq-upstream/data_platform/etl/batch_postgres_minio_etl.py", line 177, i
n run_etl
    self.load(transformed_data, bucket_name, table)
  File "/workspaces/linkedin-learning-dq-upstream/data_platform/etl/batch_postgres_minio_etl.py", line 145, i
n load
    fastavro.writer(out, schema, records)
  File "fastavro/_write.pyx", line 796, in fastavro._write.writer
  File "fastavro/_write.pyx", line 738, in fastavro._write.Writer.write
  File "fastavro/_write.pyx", line 475, in fastavro._write.write_data
  File "fastavro/_write.pyx", line 465, in fastavro._write.write_data
  File "fastavro/_write.pyx", line 409, in fastavro._write.write_record
  File "fastavro/_write.pyx", line 474, in fastavro._write.write_data
  File "fastavro/_write.pyx", line 437, in fastavro._write.write_data
  File "fastavro/_write.pyx", line 132, in fastavro._write.write_utf8
TypeError: must be string on field registration_state
```

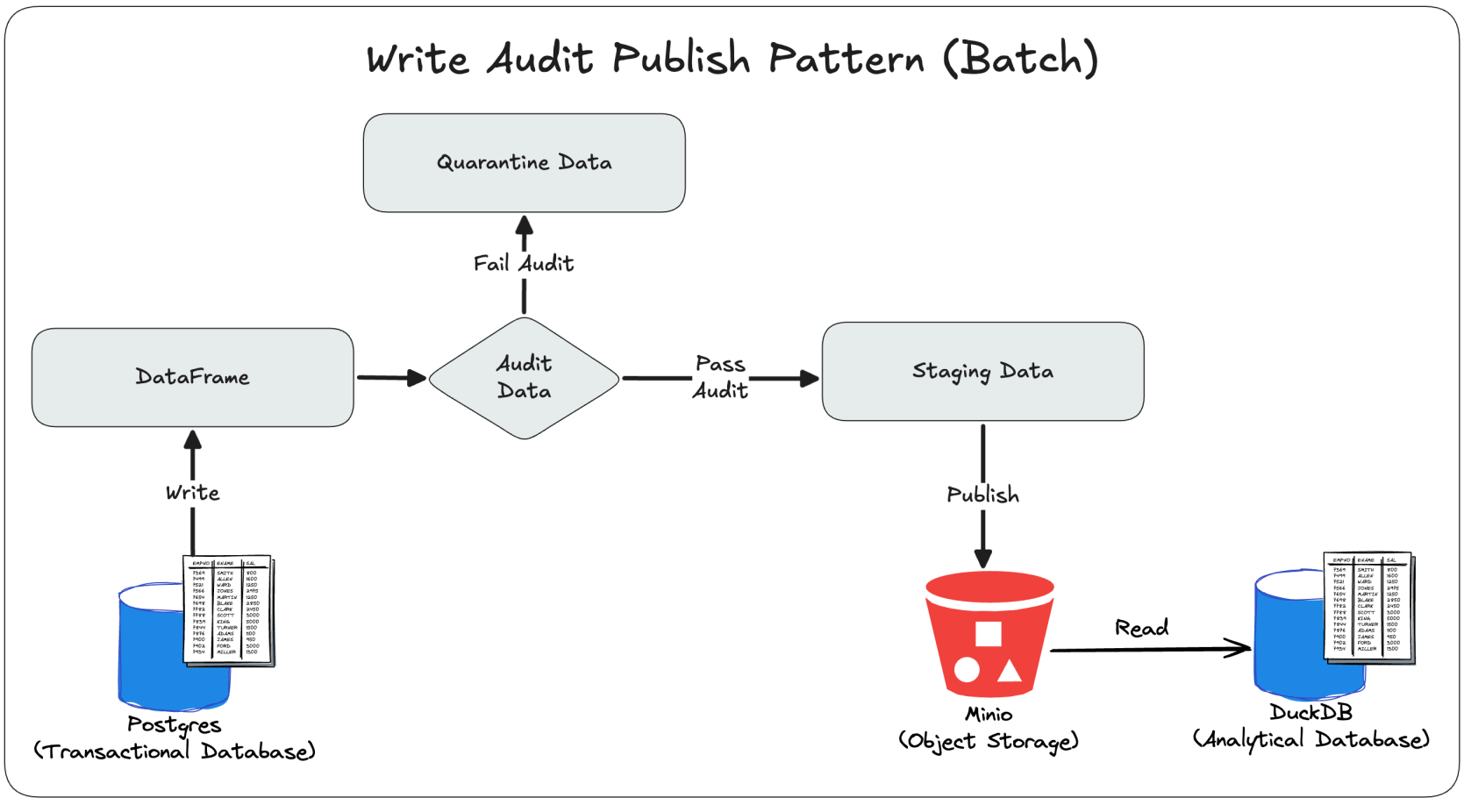
It looks like the dirty data is now causing the transformation to Avro files to fail. This has huge implications on data timeliness of the data lakehouse for downstream data consumers.

3.3 - Introduction to the Write Audit Publish (WAP) Pattern

We need to add a data quality check on the pipeline that does the following:

1. Ensures that the data pipeline doesn't fail and not fully replicate the data.
2. Flag data that will cause errors.
3. Quarantine the data for manual review later.

Thus, we need to implement the Write Audit Publish (WAP) Pattern on our replication pipeline as illustrated in the following diagram:



3.4 - Implementing the Write Audit Publish Pattern

Let's first confirm that our data is present in PostgreSQL.

In [2…

```
postgres = PostgresClient()

query = """
SELECT
    table_name
FROM
    information_schema.tables
WHERE
    table_schema = 'public'
ORDER BY
    table_name;
"""
postgres.execute_query(query, return_pd_dataframe=True)
```

Out[2…

	table_name
0	parking_violation_codes
1	parking_violations_issued

Now let's initialize the WAP class and run the WAP ETL pipeline.

In [3…

```
write_audit_publish = BatchPostgresMinioWriteAuditPublishETL(
    bucket_name="raw-data",
    schema_directory="data_platform/data/schemas"
)
write_audit_publish.run_wap()
write_audit_publish.run_etl()
```

```
INFO - data_platform.etl.batch_postgres_minio_etl - Querying PostgreSQL for public tables
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Starting Write-Audit-Publish for 2 tables: ['parking_
violations_issued', 'parking_violation_codes']
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Preparing staging and quarantine tables for 'parking_
violations_issued'
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 110000 rows from parking_violation
s_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violations_issu
ed
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violations_issued.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 36 fields for parking_violati
ons_issued
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Extracted 110000 rows from 'parking_violations_issue
d'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Populating staging table: 'parking_violations_issued'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Finished 'parking_violations_issued'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Preparing staging and quarantine tables for 'parking_
violation_codes'
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: parking_violation_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 98 rows from parking_violation_cod
es
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violation_codes.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 6 fields for parking_violatio
n_codes
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Extracted 98 rows from 'parking_violation_codes'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Populating staging table: 'parking_violation_codes'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Finished 'parking_violation_codes'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Write-Audit-Publish run complete
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Querying PostgreSQL for staging tables
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Starting ETL for 2 tables: ['staging_parking_violatio
n_codes', 'staging_parking_violations_issued']
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Processing table: staging_parking_violation_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: staging_parking_violation_cod
es
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 97 rows from staging_parking_viola
tion_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Transforming 97 records to Avro format for table: parking
_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violation_codes.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 6 fields for parking_violatio
n_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully transformed 97 records toAvro format
INFO - data_platform.etl.batch_postgres_minio_etl - Loading 97 records to MinIO bucket: raw-data
DEBUG - data_platform.etl.batch_postgres_minio_etl - Writing Avro file: /tmp/staging_parking_violation_codes_
20250927_055013.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Uploading to MinIO: postgres-exports/staging_parking_vio
lation_codes/2025/09/27/staging_parking_violation_codes_20250927_055013.avro
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "HEAD /raw-data HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /raw-data/postgres-exports/staging_parking_violation_
codes/2025/09/27/staging_parking_violation_codes_20250927_055013.avro HTTP/1.1" 200 0
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully uploaded 97 records to postgres-exports/stag
ing_parking_violation_codes/2025/09/27/staging_parking_violation_codes_20250927_055013.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Cleaned up temporary file: /tmp/staging_parking_violatio
n_codes_20250927_055013.avro
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Successfully processed table: staging_parking_violati
on_codes
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Processing table: staging_parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: staging_parking_violations_is
sued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 100000 rows from staging_parking_v
iolations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Transforming 100000 records to Avro format for table: par
king_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violations_issu
ed
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violations_issued.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 36 fields for parking_violati
ons_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully transformed 100000 records toAvro format
INFO - data_platform.etl.batch_postgres_minio_etl - Loading 100000 records to MinIO bucket: raw-data
DEBUG - data_platform.etl.batch_postgres_minio_etl - Writing Avro file: /tmp/staging_parking_violations_issue
d_20250927_055014.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Uploading to MinIO: postgres-exports/staging_parking_vio
lations_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "HEAD /raw-data HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "POST /raw-data/postgres-exports/staging_parking_violation
s_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro?uploads= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - Starting new HTTP connection (2): minio:9000
```

```
DEBUG - urllib3.connectionpool - Starting new HTTP connection (3): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /raw-data/postgres-exports/staging_parking_violations
_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro?partNumber=3&uploadId=ZjExNzMzMWUtY
jU2NC00NmU0LTg0ZTAtNWU4MjkxMjVmMDk0LjA0ZmRkYzYxLTk3NmMtNDMyZC1iNmI4LTUzMTY2YjM0MTE4MXgxNzU4OTUyMjE1NDg1NTk0NT
Uy HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /raw-data/postgres-exports/staging_parking_violations
_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro?partNumber=1&uploadId=ZjExNzMzMWUtY
jU2NC00NmU0LTg0ZTAtNWU4MjkxMjVmMDk0LjA0ZmRkYzYxLTk3NmMtNDMyZC1iNmI4LTUzMTY2YjM0MTE4MXgxNzU4OTUyMjE1NDg1NTk0NT
Uy HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /raw-data/postgres-exports/staging_parking_violations
_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro?partNumber=2&uploadId=ZjExNzMzMWUtY
jU2NC00NmU0LTg0ZTAtNWU4MjkxMjVmMDk0LjA0ZmRkYzYxLTk3NmMtNDMyZC1iNmI4LTUzMTY2YjM0MTE4MXgxNzU4OTUyMjE1NDg1NTk0NT
Uy HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "POST /raw-data/postgres-exports/staging_parking_violation
s_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro?uploadId=ZjExNzMzMWUtYjU2NC00NmU0L
Tg0ZTAtNWU4MjkxMjVmMDk0LjA0ZmRkYzYxLTk3NmMtNDMyZC1iNmI4LTUzMTY2YjM0MTE4MXgxNzU4OTUyMjE1NDg1NTk0NTUy HTTP/1.1"
200 0
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully uploaded 100000 records to postgres-exports/
staging_parking_violations_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Cleaned up temporary file: /tmp/staging_parking_violatio
ns_issued_20250927_055014.avro
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Successfully processed table: staging_parking_violati
ons_issued
INFO - data_platform.etl.batch_postgres_minio_wap_etl - ETL run complete
```

Success! Now let's confirm that the data is present in PostgreSQL.

In [3…

```
from data_platform.clients.postgres_client import PostgresClient
postgres = PostgresClient()

query = """
SELECT
    table_name
FROM
    information_schema.tables
WHERE
    table_schema = 'public'
ORDER BY
    table_name;
"""
postgres.execute_query(query, return_pd_dataframe=True)
```

Out[3…

	table_name
0	parking_violation_codes
1	parking_violations_issued
2	quarantine_parking_violation_codes
3	quarantine_parking_violations_issued
4	staging_parking_violation_codes
5	staging_parking_violations_issued

We can see that the data is present in PostgreSQL, specifically the quarantine and staging tables.

Now let's take a look at the data in the staging and quarantine tables to ensure that the data is present and the data quality checks are working as expected.

In [3…

```
sql_query = """
select * from staging_parking_violation_codes limit 10;
"""
postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Out[3…

	code	definition	manhattan_96th_st_below	all_other_areas	pg_created_at	pg_updated_at
0	1	FAILURE TO DISPLAY BUS PERMIT	515	515	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
1	2	NO OPERATOR NAM/ADD/PH DISPLAY	515	515	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
2	3	UNAUTHORIZED PASSENGER PICK-UP	515	515	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
3	4	BUS PARKING IN LOWER MANHATTAN	115	115	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
4	5	BUS LANE VIOLATION	250	250	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
5	6	OVERNIGHT TRACTOR TRAILER PKG	500	500	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
6	7	FAILURE TO STOP AT RED LIGHT	50	50	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
7	8	IDLING	115	115	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
8	9	OBSTRUCTING TRAFFIC/INTERSECT	115	115	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789
9	10	NO STOPPING-DAY/TIME LIMITS	115	115	2025-09-27 05:44:51.252789	2025-09-27 05:44:51.252789

In [3…

```
sql_query = """
select * from quarantine_parking_violation_codes limit 10;
"""
postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Out[3…

	code	definition	manhattan_96th_st_below	all_other_areas	pg_created_at	pg_updated_at
0	-99	TEST	None	None	2025-09-27 05:45:03.885774	2025-09-27 05:45:03.885774

In [3…

```
sql_query = """
select * from staging_parking_violations_issued limit 10;
"""
postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Out[3…

	summons_number	registration_state	plate_type	issue_date	violation_code	vehicle_body_type	vehicle_make	issuing_ag
0	9010912681	CA	PAS	2022-10-11	17	SUBN	FORD	
1	4858762841	NY	PAS	2023-08-21	36	4DSD	HONDA	
2	4854645684	FL	PAS	2023-07-26	36	UT	BMW	
3	9044582707	NY	PAS	2023-04-10	21	SUBN	SUBAR	
4	9041503330	NY	PAS	2023-03-21	21	4DSD	CHEVR	
5	8964996320	NY	COM	2023-06-12	21	VAN	CHEVR	
6	9019285804	NY	SRF	2022-11-20	40	SUBN	VOLKS	
7	9000744076	CT	COM	2022-07-19	69	SUBN	LINCO	
8	4839205334	NY	PAS	2023-04-30	36	SUBN	TESLA	
9	9006263692	NY	PAS	2022-09-20	16	SUBN	CHEVR	

10 rows × 36 columns

In [3…

```
sql_query = """
select * from quarantine_parking_violations_issued limit 10;
"""
postgres.execute_query(sql_query, return_pd_dataframe=True)
```

Out[3…

	summons_number	registration_state	plate_type	issue_date	violation_code	vehicle_body_type	vehicle_make	issuing_ag
0	999077846086	None	None	None	-99	None	None	
1	999077846050	None	None	None	-99	None	None	
2	999077835921	None	None	None	-99	None	None	
3	999077835209	None	None	None	-99	None	None	
4	999077165216	None	None	None	-99	None	None	
5	999077102917	None	None	None	-99	None	None	
6	999077095743	None	None	None	-99	None	None	
7	999077095299	None	None	None	-99	None	None	
8	999077055022	None	None	None	-99	None	None	
9	999077025420	None	None	None	-99	None	None	

10 rows × 36 columns

Let's now confirm that the staging data is present in the MinIO bucket too.

In [3…

```
objects = minio.list_objects("raw-data")
print("Files in raw-data bucket:")
for obj in sorted(objects):
    print(f"  {obj}")
```

```
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?delimiter=&encoding-type=url&list-type=2&ma
x-keys=1000&prefix= HTTP/1.1" 200 0
Files in raw-data bucket:
  postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_20250927_053734.avro
  postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_20250927_054008.avro
  postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_20250927_054457.avro
  postgres-exports/parking_violations_issued/2025/09/27/parking_violations_issued_20250927_053735.avro
  postgres-exports/parking_violations_issued/2025/09/27/parking_violations_issued_20250927_054008.avro
  postgres-exports/parking_violations_issued/2025/09/27/parking_violations_issued_20250927_054457.avro
  postgres-exports/staging_parking_violation_codes/2025/09/27/staging_parking_violation_codes_20250927_05501
3.avro
  postgres-exports/staging_parking_violations_issued/2025/09/27/staging_parking_violations_issued_20250927_05
5014.avro
```

Now let's run the full script end-to-end to see the results and logs!

In [3…

```
!python3 data_platform/scripts/run_wap_etl.py
```

```
INFO - __main__ - Starting ETL: Postgres→MinIO (bucket=raw-data)
INFO - data_platform.etl.batch_postgres_minio_etl - Querying PostgreSQL for public tables
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Starting Write-Audit-Publish for 2 tables: ['parking_
violations_issued', 'parking_violation_codes']
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Preparing staging and quarantine tables for 'parking_
violations_issued'
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 110000 rows from parking_violation
s_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violations_issu
ed
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violations_issued.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 36 fields for parking_violati
ons_issued
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Extracted 110000 rows from 'parking_violations_issue
d'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Populating staging table: 'parking_violations_issued'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Finished 'parking_violations_issued'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Preparing staging and quarantine tables for 'parking_
violation_codes'
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: parking_violation_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 98 rows from parking_violation_cod
es
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violation_codes.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 6 fields for parking_violatio
n_codes
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Extracted 98 rows from 'parking_violation_codes'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Populating staging table: 'parking_violation_codes'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Finished 'parking_violation_codes'
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Write-Audit-Publish run complete
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Querying PostgreSQL for staging tables
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Starting ETL for 2 tables: ['staging_parking_violatio
n_codes', 'staging_parking_violations_issued']
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Processing table: staging_parking_violation_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: staging_parking_violation_cod
es
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 97 rows from staging_parking_viola
tion_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Transforming 97 records to Avro format for table: parking
_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violation_codes.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violation_codes
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 6 fields for parking_violatio
n_codes
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully transformed 97 records toAvro format
INFO - data_platform.etl.batch_postgres_minio_etl - Loading 97 records to MinIO bucket: data-lake
DEBUG - data_platform.etl.batch_postgres_minio_etl - Writing Avro file: /tmp/staging_parking_violation_codes_
20250927_055235.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Uploading to MinIO: postgres-exports/staging_parking_vio
lation_codes/2025/09/27/staging_parking_violation_codes_20250927_055235.avro
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /data-lake?location= HTTP/1.1" 404 0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /data-lake HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /data-lake/postgres-exports/staging_parking_violation
_codes/2025/09/27/staging_parking_violation_codes_20250927_055235.avro HTTP/1.1" 200 0
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully uploaded 97 records to postgres-exports/stag
ing_parking_violation_codes/2025/09/27/staging_parking_violation_codes_20250927_055235.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Cleaned up temporary file: /tmp/staging_parking_violatio
n_codes_20250927_055235.avro
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Successfully processed table: staging_parking_violati
on_codes
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Processing table: staging_parking_violations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Extracting data from table: staging_parking_violations_is
sued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully extracted 100000 rows from staging_parking_v
iolations_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Transforming 100000 records to Avro format for table: par
king_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Inferring Avro schema for table: parking_violations_issu
ed
DEBUG - data_platform.etl.batch_postgres_minio_etl - Loading schema from: data_platform/data/schemas/parking_
violations_issued.json
DEBUG - data_platform.etl.batch_postgres_minio_etl - Successfully loaded schema for parking_violations_issued
DEBUG - data_platform.etl.batch_postgres_minio_etl - Generated Avro schema with 36 fields for parking_violati
ons_issued
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully transformed 100000 records toAvro format
INFO - data_platform.etl.batch_postgres_minio_etl - Loading 100000 records to MinIO bucket: data-lake
DEBUG - data_platform.etl.batch_postgres_minio_etl - Writing Avro file: /tmp/staging_parking_violations_issue
d_20250927_055236.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Uploading to MinIO: postgres-exports/staging_parking_vio
lations_issued/2025/09/27/staging_parking_violations_issued_20250927_055236.avro
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /data-lake?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "HEAD /data-lake HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "POST /data-lake/postgres-exports/staging_parking_violatio
ns_issued/2025/09/27/staging_parking_violations_issued_20250927_055236.avro?uploads= HTTP/1.1" 200 0
```

```
DEBUG - urllib3.connectionpool - Starting new HTTP connection (2): minio:9000
DEBUG - urllib3.connectionpool - Starting new HTTP connection (3): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /data-lake/postgres-exports/staging_parking_violation
s_issued/2025/09/27/staging_parking_violations_issued_20250927_055236.avro?partNumber=3&uploadId=ZjExNzMzMWUt
YjU2NC00NmU0LTg0ZTAtNWU4MjkxMjVmMDk0LmFiZGU3MGI4LTk4OTMtNGUxNS04NWQ0LTdmODkzNDE3MTllZXgxNzU4OTUyMzU3NDA4ODc0M
Dkw HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /data-lake/postgres-exports/staging_parking_violation
s_issued/2025/09/27/staging_parking_violations_issued_20250927_055236.avro?partNumber=1&uploadId=ZjExNzMzMWUt
YjU2NC00NmU0LTg0ZTAtNWU4MjkxMjVmMDk0LmFiZGU3MGI4LTk4OTMtNGUxNS04NWQ0LTdmODkzNDE3MTllZXgxNzU4OTUyMzU3NDA4ODc0M
Dkw HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "PUT /data-lake/postgres-exports/staging_parking_violation
s_issued/2025/09/27/staging_parking_violations_issued_20250927_055236.avro?partNumber=2&uploadId=ZjExNzMzMWUt
YjU2NC00NmU0LTg0ZTAtNWU4MjkxMjVmMDk0LmFiZGU3MGI4LTk4OTMtNGUxNS04NWQ0LTdmODkzNDE3MTllZXgxNzU4OTUyMzU3NDA4ODc0M
Dkw HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "POST /data-lake/postgres-exports/staging_parking_violatio
ns_issued/2025/09/27/staging_parking_violations_issued_20250927_055236.avro?uploadId=ZjExNzMzMWUtYjU2NC00NmU0
LTg0ZTAtNWU4MjkxMjVmMDk0LmFiZGU3MGI4LTk4OTMtNGUxNS04NWQ0LTdmODkzNDE3MTllZXgxNzU4OTUyMzU3NDA4ODc0MDkw HTTP/1.
1" 200 0
INFO - data_platform.etl.batch_postgres_minio_etl - Successfully uploaded 100000 records to postgres-exports/
staging_parking_violations_issued/2025/09/27/staging_parking_violations_issued_20250927_055236.avro
DEBUG - data_platform.etl.batch_postgres_minio_etl - Cleaned up temporary file: /tmp/staging_parking_violatio
ns_issued_20250927_055236.avro
INFO - data_platform.etl.batch_postgres_minio_wap_etl - Successfully processed table: staging_parking_violati
ons_issued
INFO - data_platform.etl.batch_postgres_minio_wap_etl - ETL run complete
INFO - __main__ - Completed ETL: Postgres→MinIO
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?delimiter=&encoding-type=url&list-type=2&ma
x-keys=1000&prefix= HTTP/1.1" 200 0
INFO - __main__ - minio:raw-data/postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_
20250927_053734.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_
20250927_054008.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violation_codes/2025/09/27/parking_violation_codes_
20250927_054457.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violations_issued/2025/09/27/parking_violations_iss
ued_20250927_053735.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violations_issued/2025/09/27/parking_violations_iss
ued_20250927_054008.avro
INFO - __main__ - minio:raw-data/postgres-exports/parking_violations_issued/2025/09/27/parking_violations_iss
ued_20250927_054457.avro
INFO - __main__ - minio:raw-data/postgres-exports/staging_parking_violation_codes/2025/09/27/staging_parking_
violation_codes_20250927_055013.avro
INFO - __main__ - minio:raw-data/postgres-exports/staging_parking_violations_issued/2025/09/27/staging_parkin
g_violations_issued_20250927_055014.avro
INFO - data_platform.etl.batch_minio_duckdb_elt - Extracting latest Avro for table 'staging_parking_violation
_codes' from 'postgres-exports'
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?delimiter=&encoding-type=url&list-type=2&ma
x-keys=1000&prefix= HTTP/1.1" 200 0
INFO - data_platform.etl.batch_minio_duckdb_elt - Found 1 candidates, latest: postgres-exports/staging_parkin
g_violation_codes/2025/09/27/staging_parking_violation_codes_20250927_055013.avro
INFO - data_platform.etl.batch_minio_duckdb_elt - Loading 's3://raw-data/postgres-exports/staging_parking_vio
lation_codes/2025/09/27/staging_parking_violation_codes_20250927_055013.avro' into staging.staging_parking_vi
olation_codes
DEBUG - data_platform.etl.batch_minio_duckdb_elt - Creating staging schema
INFO - data_platform.etl.batch_minio_duckdb_elt - Creating table staging.staging_parking_violation_codes
INFO - data_platform.etl.batch_minio_duckdb_elt - Successfully loaded 97 rows into staging.staging_parking_vi
olation_codes
INFO - __main__ - Completed ELT for table=staging_parking_violation_codes
INFO - data_platform.etl.batch_minio_duckdb_elt - Extracting latest Avro for table 'staging_parking_violation
s_issued' from 'postgres-exports'
DEBUG - urllib3.connectionpool - Starting new HTTP connection (1): minio:9000
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?location= HTTP/1.1" 200 0
DEBUG - urllib3.connectionpool - http://minio:9000 "GET /raw-data?delimiter=&encoding-type=url&list-type=2&ma
x-keys=1000&prefix= HTTP/1.1" 200 0
INFO - data_platform.etl.batch_minio_duckdb_elt - Found 1 candidates, latest: postgres-exports/staging_parkin
g_violations_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro
INFO - data_platform.etl.batch_minio_duckdb_elt - Loading 's3://raw-data/postgres-exports/staging_parking_vio
lations_issued/2025/09/27/staging_parking_violations_issued_20250927_055014.avro' into staging.staging_parkin
g_violations_issued
DEBUG - data_platform.etl.batch_minio_duckdb_elt - Creating staging schema
INFO - data_platform.etl.batch_minio_duckdb_elt - Creating table staging.staging_parking_violations_issued
INFO - data_platform.etl.batch_minio_duckdb_elt - Successfully loaded 100000 rows into staging.staging_parkin
g_violations_issued
INFO - __main__ - Completed ELT for table=staging_parking_violations_issued
INFO - __main__ - DuckDB staging tables:
    database   schema                                name
0  lakehouse  staging            parking_violation_codes
1  lakehouse  staging          parking_violations_issued
2  lakehouse  staging    staging_parking_violation_codes
3  lakehouse  staging  staging_parking_violations_issued
```

Success! The logs confirm the pipeline works, and we see both the original raw data in DuckDB and the new staging data from the WAP pipeline.

Finally, let's confirm that the raw data and the staging data result in the same report.

In [3…

```
query = """
SELECT
    parking_violation_codes.code,
    parking_violation_codes.definition,
    COUNT(parking_violations_issued.violation_code) AS ticket_count
FROM
    staging.parking_violation_codes
JOIN
    staging.parking_violations_issued ON
    parking_violation_codes.code = parking_violations_issued.violation_code
GROUP BY
    parking_violation_codes.code,
    parking_violation_codes.definition
ORDER BY
    ticket_count DESC
LIMIT 10;
"""
with DuckdbClient(db_path="data_platform/lakehouse/lakehouse.db") as duckdb:
    display(duckdb.execute_query(query, return_pd_dataframe=True))
```

	code	definition	ticket_count
0	36	PHTO SCHOOL ZN SPEED VIOLATION	37064
1	21	NO PARKING-STREET CLEANING	12149
2	38	FAIL TO DSPLY MUNI METER RECPT	6482
3	14	NO STANDING-DAY/TIME LIMITS	5047
4	7	FAILURE TO STOP AT RED LIGHT	4055
5	20	NO PARKING-DAY/TIME LIMITS	3949
6	40	FIRE HYDRANT	3902
7	71	INSP. STICKER-EXPIRED/MISSING	3655
8	5	BUS LANE VIOLATION	3477
9	70	REG. STICKER-EXPIRED/MISSING	2595

In [3…

```
query = """
SELECT
    staging_parking_violation_codes.code,
    staging_parking_violation_codes.definition,
    COUNT(staging_parking_violations_issued.violation_code) AS ticket_count
FROM
    staging.staging_parking_violation_codes
JOIN
    staging.staging_parking_violations_issued ON
    staging_parking_violation_codes.code = staging_parking_violations_issued.violation_code
GROUP BY
    staging_parking_violation_codes.code,
    staging_parking_violation_codes.definition
ORDER BY
    ticket_count DESC
LIMIT 10;
"""
with DuckdbClient(db_path="data_platform/lakehouse/lakehouse.db") as duckdb:
    display(duckdb.execute_query(query, return_pd_dataframe=True))
```

	code	definition	ticket_count
0	36	PHTO SCHOOL ZN SPEED VIOLATION	37064
1	21	NO PARKING-STREET CLEANING	12149
2	38	FAIL TO DSPLY MUNI METER RECPT	6482
3	14	NO STANDING-DAY/TIME LIMITS	5047
4	7	FAILURE TO STOP AT RED LIGHT	4055
5	20	NO PARKING-DAY/TIME LIMITS	3949
6	40	FIRE HYDRANT	3902
7	71	INSP. STICKER-EXPIRED/MISSING	3655
8	5	BUS LANE VIOLATION	3477
9	70	REG. STICKER-EXPIRED/MISSING	2595

Success!

3.5 - Replication Code Exploration

Take a moment to explore the replication code. One thing to note is that the Write Audit Publish pattern can also be used for streaming data where it's often the most popular (but outside the scope of this course).

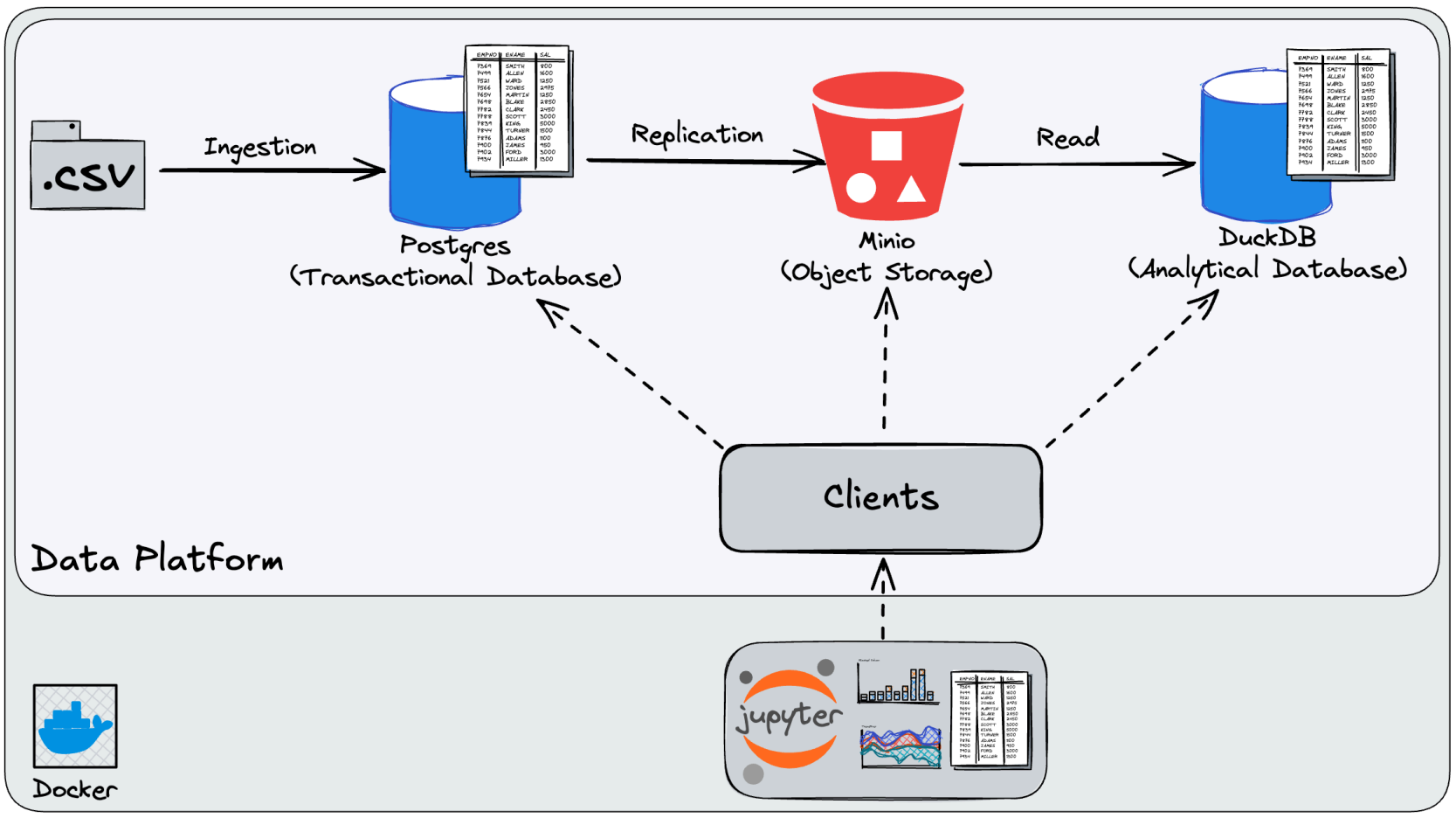
Chapter 4 - Data Platform Product Requirements Document (PRD)

We now have all of the information we need to create the Product Requirements Document (PRD).

Below is the PRD outline to fill out for the data platform. If you want an extra challenge, take a moment to try and fill it out yourself before watching the video where we can compare what we wrote.

1. Data Platform Architecture Overview

Current Architecture



The platform centers on a PostgreSQL transactional database for ingestion and storage of operational datasets, a MinIO object store acting as the data lake, and DuckDB as a lightweight lakehouse/analytics engine. Ingestion loads CSVs into PostgreSQL with schema-driven table creation, primary keys, and idempotent upserts. Replication exports tables from PostgreSQL to MinIO as Avro files, partitioned by date-like path components, and DuckDB reads the latest Avro artifact into a `staging` schema for analysis.

A Write–Audit–Publish (WAP) flow is available: rows first land in `staging_*` tables, failing rows are redirected to `quarantine_*` tables, and only audited, conforming data is published downstream. End-to-end execution is orchestrated via simple runner scripts.

Key Components:

- **Ingestion (CSV → PostgreSQL)**: `data_platform/scripts/run_ingestion.py` → `data_platform/ingestion/postgres_ingestion_csv.py`
- **Clients**: `data_platform/clients/postgres_client.py`, `data_platform/clients/minio_client.py`, `data_platform/clients/duckdb_client.py`
- **Configs**: `data_platform/config/postgres_config.py`, `data_platform/config/minio_config.py`, `data_platform/config/duckdb_config.py`
- **Replication (PostgreSQL → MinIO Avro)**: `data_platform/etl/batch_postgres_minio_etl.py`, `data_platform/scripts/run_etl.py`
- **Lakehouse ELT (MinIO Avro → DuckDB)**: `data_platform/etl/batch_minio_duckdb_elt.py`
- **WAP ETL**: `data_platform/etl/batch_postgres_minio_wap_etl.py`, `data_platform/scripts/run_wap_etl.py`
- **Transactions Simulator**: `data_platform/transactions/concurrency_simulator.py`
- **Data, Schemas, and Samples**: `data_platform/data/**`

2. Gap Analysis

2.1 Ingestion into Transactional Database

Here's the current state in markdown format:

Current State:

- **Schema validation**: Uses JSON schema files to define table structure with specific data types
- **Data cleaning**: Automatically removes empty rows and standardizes column names
- **Duplicate prevention**: Implements UPSERT operations using `ON CONFLICT` to handle duplicate primary keys
- **Audit trails**: Adds automatic `pg_created_at` and `pg_updated_at` timestamps with update triggers

- **Idempotent processing**: Can run the same ingestion multiple times without creating duplicates
- **Type safety**: Enforces PostgreSQL data types defined in schema (BIGINT, TEXT, DATE, INTEGER, etc.)
- **Primary key constraints**: Ensures data uniqueness through primary key definitions
- **Comprehensive logging**: Detailed logging at DEBUG, INFO, and ERROR levels for troubleshooting
- **Transaction safety**: Uses atomic database transactions to ensure all data operations complete entirely or not at all, preventing partial data corruption
- **Error handling**: Validates file existence and includes proper exception handling

Gaps Found:

- **Business rule constraints missing**: Negative or invalid values can pass schema typing. Example observed in the notebook: `parking_violation_codes.code = -99` with definition `TEST` ingested successfully, then surfaced in reports.
- **No referential integrity between facts and dimensions**: `parking_violations_issued.violation_code` is not enforced to exist in `parking_violation_codes.code`, permitting orphan codes and inconsistent joins.
- **Over-permissive text fields**: Fields like `vehicle_expiration_date` are `TEXT` and contain placeholders like `0` or `88888888`, which are semantically invalid dates and propagate downstream.
- **Nullability not curated for downstream**: Many columns permit NULLs; later stages (Avro writing) may expect integers and fail when NULLs appear for integer fields.
- **Ingestion lacks quarantine at the source**: Validation is primarily type/shape oriented; domain violations are only caught later in WAP or analytics, not at the initial landing.

Recommendations:

- **Add database CHECK constraints** to enforce domain rules at write time. Proven examples from the notebook:
  - `ALTER TABLE parking_violation_codes ADD CONSTRAINT code_check CHECK (code >= 0);`
  - `ALTER TABLE parking_violations_issued ADD CONSTRAINT summons_number_is_ten_digits CHECK (summons_number::text ~ '^[0-9]{10}$');`
- **Add foreign key constraints** to maintain referential integrity:
  - `ALTER TABLE parking_violations_issued ADD CONSTRAINT fk_violation_code FOREIGN KEY (violation_code) REFERENCES parking_violation_codes(code);`
- **Tighten data types and casting**:
  - Store true dates in `DATE` columns (e.g., normalize `vehicle_expiration_date` or persist as NULL when invalid).
  - Ensure numeric fine columns are `INTEGER` and default to NULL if missing, not strings.
- **Introduce ingestion-time quarantine**:
  - Capture rows that fail constraints into `quarantine_*` tables with error reasons, not just fail the batch.
- **Extend JSON schemas with rule hints** (ranges, regex) and surface them in ingestion validations, not only in DDL.
- **Document allowed enumerations** (e.g., `registration_state`, `plate_type`) and validate against them during ingestion.

2.2 Transactions on Transactional Database

Current State: Concurrent writers can operate at PostgreSQL's default `READ COMMITTED` isolation. The `TransactionSimulator` demonstrates nondeterministic totals under contention when two workers update the same logical record without coordination.

In the notebook, repeated runs of `simulator.simulate(increment_a=1, increment_b=2, iterations=5)` produced different results at default isolation. When the isolation was set to `SERIALIZABLE`, results stabilized at the expected total (15), with transient retries on serialization failures.

Gaps Found:

- **Potential for lost updates/serialization anomalies** under default isolation and concurrent writers.
- **No standardized retry policy** for serialization failures across write paths.
- **No explicit locking pattern** for critical read-modify-write sections.

Recommendations:

- **Use `SERIALIZABLE` isolation** for high-risk transactional sections or adopt `SELECT ... FOR UPDATE` to serialize access to hot rows.
- **Implement bounded retries** on `could not serialize access due to concurrent update` with jitter backoff (the simulator retried up to 4x successfully).
- **Add metrics/logging** for serialization failures and retries to tune contention handling.

2.3 Replication to Data Lakehouse

Current State: `batch_postgres_minio_etl.py` exports each public PostgreSQL table to Avro and writes to MinIO under date-partitioned keys (e.g., `postgres-exports/<table>/YYYY/MM/DD/<table>_timestamp.avro`). `batch_minio_duckdb_elt.py` then reads the latest object per table into DuckDB `staging.<table>`. The WAP pipeline (`batch_postgres_minio_wap_etl.py`) writes to `staging_*` and `quarantine_*` tables in PostgreSQL before exporting curated Avro snapshots.

Gaps Found:

- **Type nullability mismatch caused ETL failure**: Avro writer raised `TypeError: an integer is required on field manhattan_96th_st_below` when NULLs were present for an `int` field.

- **Schema evolution and contracts are implicit**: Schemas are inferred from JSON table schemas; Avro unions for nullability aren't consistently declared.
- **Publishing policy not uniform**: The basic ETL writes all tables; only the WAP flow enforces per-row audits/quarantine before publish.

Recommendations:

- **Define explicit Avro schemas with unions for optional fields**, e.g., `["null", "long"]` for integer columns that can be missing.
- **Coerce and validate types in transforms**: Convert placeholder strings like `""`, `"0"`, `"88888888"` to NULLs before serialization.
- **Standardize on WAP for production**: Always stage and audit; only publish if checks pass, while retaining quarantined records with reasons.
- **Retention and snapshotting**: Keep multiple dated exports (already present) and implement lifecycle policies to manage storage.

3. Improvement Suggestions

Priority 1 (High Impact)

Improvement: Enforce domain integrity and referential integrity in PostgreSQL

- **What**: Add CHECKs (e.g., `code >= 0`, 10-digit `summons_number`) and FKs (`violations_issued.violation_code → violation_codes.code`).
- **Why**: Prevent bad data from entering the system; ensures consistent joins and analytics.
- **Tradeoffs**: Stricter writes may initially reject legacy rows; requires migration/backfill to conform.
- **Test Results**: After adding `code_check`, attempts to ingest negative codes failed with:

```
CheckViolation: new row for relation "parking_violation_codes" violates check constraint "code_chec
k"
DETAIL:  Failing row contains (-99, TEST, null, null, ...).
```

Priority 1 (High Impact)

Improvement: Adopt Write–Audit–Publish (WAP) as the default replication pattern

- **What**: Land rows in `staging_*`, route invalid rows to `quarantine_*`, export only staged/passed data to MinIO and DuckDB.
- **Why**: Prevents broken snapshots and propagating bad records; keeps explainability via quarantined examples.
- **Tradeoffs**: More tables and control flow to manage; slightly more compute/storage.
- **Test Results**: With WAP, valid tables exported and loaded; invalid rows were visible in `quarantine_*` and did not break downstream ELT.

Priority 2 (Medium Impact)

Improvement: Make Avro schemas explicitly nullable for optional numeric fields

- **What**: Change Avro types from `long → ["null","long"]` and ensure transforms map missing/invalid values to NULL.
- **Why**: Prevent writer failures like `TypeError: an integer is required on field manhattan_96th_st_below`.
- **Tradeoffs**: Consumers must handle NULLs; contract needs documentation.
- **Test Results**: Expected to resolve the observed `fastavro` error seen during `run_etl.py`.

Priority 2 (Medium Impact)

Improvement: Strengthen transaction semantics for hot updates

- **What**: Use `SERIALIZABLE` isolation or row-level locks with bounded retries for conflicting writers.
- **Why**: Eliminates lost updates and nondeterministic outcomes.
- **Tradeoffs**: Higher contention may increase retries/latency; needs careful scoping.
- **Test Results**: Simulator produced stable, expected totals (15) with `SERIALIZABLE` and retries.

Priority 3 (Low Impact)

Improvement: Enumerations and data dictionaries for key fields

- **What**: Validate `registration_state`, `plate_type`, `issuing_agency` against known sets or lookup tables.
- **Why**: Reduces downstream surprises and improves data quality SLAs.
- **Tradeoffs**: Requires curation and periodic updates.
- **Test Results**: N/A (design enhancement).

4. Remaining Questions

- **How should incremental ingestion and CDC be handled?** Current jobs are batch-oriented; deletes/updates are not explicitly modeled.

- **What is the authoritative contract for Avro schemas?** Should we version and publish them, or continue to infer from table schemas?
- **Who owns quarantine triage and remediation SLAs?** Define ownership, turnaround, and auto-retry policies.
- **Are there PII/governance requirements?** If so, classify fields and apply masking/access controls.
- **What are SLOs for freshness and failure budgets?** Define alerting and error budgets for ingestion and replication.

5. Next Steps

Immediate Actions:

- Add PostgreSQL constraints: `code_check`, `summons_number_is_ten_digits`, and `fk_violation_code`.
- Update Avro schema generation in `batch_postgres_minio_etl.py` to emit nullable types for optional integers and coerce invalid strings to NULL.
- Default to WAP: run `data_platform/scripts/run_wap_etl.py` in place of the basic ETL for routine replication.
- Document enumerations and add validations in `postgres_ingestion_csv.py`.

Future Considerations:

- Introduce CDC/incremental loads; consider Debezium or logical replication for near-real-time export.
- Automate schema/version management and contract testing for Avro.
- Add data quality dashboards and alerts (row rejection rates, serialization retries, latency).
- Evaluate partitioning, lifecycle, and cost management policies in MinIO buckets.

File References (for reviewers)

- `data_platform/ingestion/postgres_ingestion_csv.py`
- `data_platform/clients/postgres_client.py`
- `data_platform/config/postgres_config.py`
- `data_platform/scripts/run_ingestion.py`
- `data_platform/etl/batch_postgres_minio_etl.py`
- `data_platform/etl/batch_minio_duckdb_elt.py`
- `data_platform/etl/batch_postgres_minio_wap_etl.py`
- `data_platform/scripts/run_etl.py`
- `data_platform/scripts/run_wap_etl.py`
- `data_platform/transactions/concurrency_simulator.py`
- `data_platform/clients/minio_client.py`
- `data_platform/clients/duckdb_client.py`
- `data_platform/data/schemas/*.json`

END OF PROJECT WALKTHROUGH